

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA
UNIVERSITATEA TEHNICĂ A MOLDOVEI
FACULTATEA CALCULATOARE, INFORMATICĂ ȘI MICROELECTRONICĂ
DEPARTAMENTUL INGINERIA SOFTWARE ȘI AUTOMATICĂ
SPECIALITATEA TEHNOLOGIA INFORMAȚIEI



RAPORT

la proiectul de an

Tehnici Moderne de Programare pentru Platforme

**Tema: Aplicarea pattern-urilor GoF în dezvoltarea
unei platforme de comerț electronic Kinder**

A elaborat:

st. gr. TI-235 Chircu Mihail

A verificat:

lect. univ. Mihai Musteață

Chișinău – 2026

CUPRINS

ABREVIERI	3
INTRODUCERE	4
1 ANALIZA DOMENIULUI DE STUDIU	5
1.1 Tema și scopul proiectului	5
1.2 Contextul design patterns în inginerie software	6
1.3 Platforma Laravel ca mediu de aplicare	6
1.4 Obiectivele proiectului	7
2 PROIECTAREA APLICAȚIEI	8
2.1 Arhitectura generală Kinder	8
2.2 Pattern-uri creaționale	9
2.3 Pattern-uri structurale	14
2.4 Pattern-uri comportamentale	20
3 REALIZAREA APLICAȚIEI	29
3.1 Stack tehnologic	29
3.2 Fluxul de checkout	30
3.3 Panoul administrativ	32
3.4 Extensibilitatea demonstrată	34
CONCLUZII	35
BIBLIOGRAFIE	37
ANEXA A Repository GitHub	38
ANEXA B Fragmente de cod reprezentativ	39

ABREVIERI

GoF — Gang of Four — cei patru autori (Gamma, Helm, Johnson, Vlissides) ai cărții „Design Patterns: Elements of Reusable Object-Oriented Software" (1994)

UML — Unified Modeling Language — limbaj standard de modelare pentru sisteme software

SOLID — Ansamblul a cinci principii OOP: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion

DI — Dependency Injection — tehnică prin care dependențele unui obiect sunt furnizate din afară

IoC — Inversion of Control — principiu conform căruia controlul execuției este delegat unui framework

ORM — Object-Relational Mapping — tehnică de mapare între obiecte software și rânduri în bază de date

MVC — Model-View-Controller — pattern arhitectural folosit de Laravel

DTO — Data Transfer Object — obiect care transportă date între procese fără logică de business

API — Application Programming Interface

SKU — Stock Keeping Unit — cod unic de identificare a unui produs

CRUD — Create, Read, Update, Delete — operațiile standard pe o entitate

HTTP — HyperText Transfer Protocol

TTL — Time To Live — durata de viață a unei intrări în cache

MDL — Leu moldovenesc — moneda Republicii Moldova

TVA — Taxa pe Valoarea Adăugată

CSV — Comma-Separated Values — format text tabular

JSON — JavaScript Object Notation — format text de schimb de date

SMS — Short Message Service

DB — Database — bază de date

IBAN — International Bank Account Number

INTRODUCERE

Dezvoltarea software modernă pune în fața echipelor de inginerie problema repetitivă de a rezolva clase asemănătoare de probleme. Obiectele complexe trebuie construite pas cu pas, subsistemele independente trebuie orchestrate într-un flux coerent, algoritmi interschimbabili trebuie injectați la runtime fără cuplare strânsă, iar reacțiile la evenimente trebuie distribuite către mai mulți consumatori fără a modifica sursa. La aceste probleme au răspuns în 1994 cei patru autori cunoscuți ca Gang of Four prin cartea „Design Patterns: Elements of Reusable Object-Oriented Software”, care a catalogat douăzeci și trei de soluții reutilizabile devenite între timp vocabular comun al profesiei.

Prezentul proiect de an își propune să aplice pattern-urile GoF la un caz real de dezvoltare — o platformă de comerț electronic pentru jucării copiilor, numită Kinder. Alegerea domeniului nu este arbitrară: comerțul electronic conține în mod natural complexitatea necesară pentru a valida alegerea fiecărui pattern, fără a recurge la exemple artificiale. Construcția unei comenzi este un proces cu cincisprezece câmpuri și logică tranzacțională (Builder). Catalogul de produse cere sortare interschimbabilă (Strategy) și caching peste query-uri grele (Proxy). Checkout-ul orchestrează șase subsisteme (Facade), integrează gateway-uri de plată cu API-uri diferite (Adapter) și declanșează reacții post-comandă decuplate (Observer). Fiecare pattern găsește o problemă potrivită, nu o problemă construită pentru a-l justifica.

Lucrarea este organizată în trei capitole. Capitolul întâi descrie contextul teoretic și prezintă tema, scopul și obiectivele. Capitolul al doilea se concentrează pe proiectarea aplicației, expunând pe rând cele cincisprezece pattern-uri GoF implementate — patru creaționale, cinci structurale și șase comportamentale — împreună cu diagramele UML corespunzătoare și descrierea rolurilor componentelor. Capitolul al treilea ilustrează realizarea practică prin capturi de ecran ale fluxurilor principale și fragmente de cod reprezentativ. Concluziile recapitulează contribuțiile, bibliografia listează sursele consultate, iar anexele oferă linkul către repository-ul public și fragmente suplimentare de cod.

Rezultatul este o aplicație funcțională, a cărei arhitectură demonstrează că pattern-urile GoF, aplicate cu discernământ, rămân la fel de relevante astăzi ca la momentul publicării lor — nu ca decorație, ci ca instrumente care rezolvă probleme reale și care permit extinderea sistemului fără modificarea codului existent.

1 ANALIZA DOMENIULUI DE STUDIU

1.1 Tema și scopul proiectului

Tema proiectului de an este aplicarea pattern-urilor GoF în dezvoltarea unei platforme de comerț electronic destinată vânzării de jucării pentru copii. Scopul central al lucrării este demonstrarea practică a modului în care pattern-urile clasice din ingineria software rezolvă probleme reale de arhitectură într-o aplicație web modernă, construită pe framework-ul Laravel 11 și baza de date SQLite.

Aplicația realizată, numită „Kinder”, oferă întregul flux tipic unui magazin online: un catalog de produse organizat ierarhic pe categorii, sistem de căutare și filtrare, pagini de detaliu cu galerie de imagini, coș de cumpărături cu istoric și funcționalitate de anulare a ultimei operații, checkout cu trei metode de livrare și trei metode de plată, notificări multi-canal către client și panou administrativ complet pentru managementul produselor, categoriilor, comenzilor, reducerilor și utilizatorilor.

Dincolo de funcționalitatea vizibilă, proiectul urmărește un obiectiv didactic: fiecare componentă nebanală a sistemului a fost construită astfel încât să ilustreze aplicarea unui pattern GoF specific, ales nu pentru a bifa un item de checklist, ci pentru că rezolvă efectiv o problemă de design. În acest sens, proiectul se diferențiază de exemplele academice artificiale (de tipul Shape → Circle/Square sau Animal → Dog/Cat) prin faptul că pattern-urile sunt integrate într-un sistem funcțional al cărui domeniu generează organic motivele folosirii lor.

1.2 Contextul design patterns în inginerie software

Noțiunea de design pattern a fost preluată în software-engineering din arhitectura fizică, unde fusese dezvoltată de Christopher Alexander în anii 1970. Ideea centrală: anumite probleme recurente admit soluții recurente, iar fixarea acestora în vocabular face comunicarea între profesioniști mai rapidă și mai precisă. În 1994, Erich Gamma, Richard Helm, Ralph Johnson și John Vlissides — cunoscuți ca Gang of Four — au publicat „Design Patterns: Elements of Reusable Object-Oriented Software”, catalogând douăzeci și trei de pattern-uri grupate în trei categorii: creaționale (preocupate de instanțierea obiectelor), structurale (compunerea obiectelor în structuri mai mari) și comportamentale (distribuirea responsabilităților și fluxul de control între obiecte).

Folosirea pattern-urilor nu este un scop în sine. Un pattern aplicat unde nu e nevoie devine „over-engineering” — o complexitate gratuită care face codul mai greu de înțeles. De exemplu, pattern-ul Builder folosit doar pentru a seta câteva câmpuri simple nu rezolvă nicio problemă reală; devine o formă complicată de setter. Invers, pattern-ul Factory Method aplicat unui arbore de clase concrete care ar trebui să fie în realitate rânduri într-o bază de date (tipul de produs, culoarea, mărimea) este „unrealistic” — introduce clase care vor trebui create manual pentru fiecare variantă nouă, în loc să permită datelor să determine comportamentul.

Criteriul bun pentru utilizarea unui pattern este corespondența între structura problemei și structura soluției. Pattern-ul trebuie să aibă suficiente variante concrete pentru ca polimorfismul să fie real (două-trei strategii, doi-trei handleri), iar sursa variației trebuie să fie comportamentul, nu datele. Proiectul Kinder a fost conceput respectând aceste criterii.

1.3 Platforma Laravel ca mediu de aplicare

Laravel 11 este un framework PHP de tip MVC (Model-View-Controller) cu container de dependențe propriu, ORM Eloquent, sistem de rutare elegant și un ecosistem bogat de pachete. Alegerea Laravel pentru acest proiect nu este întâmplătoare: framework-ul este în sine o demonstrație practică a mai multor pattern-uri GoF. Facade-urile Laravel sunt implementări directe ale pattern-ului Facade. Service container-ul folosește pattern-ul Abstract Factory pentru rezolvarea dependențelor. Middleware-urile formează un Chain of Responsibility. Event-urile și Listeners-urile materializează Observer-ul. ORM-ul Eloquent înglobează Active Record, dar expune și Query Builder-ul pentru compunerea cererilor — o formă de Builder.

Dezvoltatorul care folosește Laravel beneficiază deci de o zonă de confort conceptual, putând aplica pattern-uri similare la nivelul aplicației sale cu garanția că infrastructura nu se va opune. În proiectul Kinder, integrarea pattern-urilor proprii cu cele existente în framework a fost naturală — de exemplu, evenimentul `OrderPlaced` se înregistrează prin mecanismul standard al Laravel (`Event::listen`), iar caching-ul în pattern-ul Proxy folosește facade-ul `Cache` care, la rândul său, este tot un pattern Facade.

Pentru baza de date s-a ales SQLite pentru simplitatea deployment-ului, iar pentru frontend Tailwind CSS 3 și Vite. Aceste alegeri nu sunt dictate de pattern-uri, dar afectează limitativ exemplele: vizualizările sunt generate server-side cu template-urile Blade ale Laravel, nu prin SPA, ceea ce păstrează codul simplu și focusat pe arhitectura back-end unde locuiesc pattern-urile studiate.

1.4 Obiectivele proiectului

Obiectivele concrete ale proiectului sunt următoarele:

- proiectarea și implementarea unei platforme de e-commerce funcționale, cu catalog, coș, checkout, notificări și panou administrativ;
- aplicarea a minimum douăsprezece pattern-uri GoF, câte patru din fiecare categorie (creaționale, structurale, comportamentale), conform cerințelor disciplinei;
- depășirea minimului prin implementarea a cincisprezece pattern-uri reale, selectate pentru a corespunde unor probleme autentice ale domeniului;
- ilustrarea fiecărui pattern prin diagrame UML și descrierea rolurilor componentelor, astfel încât corespondența între teorie și cod să fie evidentă;

- demonstrarea extensibilității arhitecturii prin adăugarea ulterioară a unor variante noi (al treilea canal de notificare, al patrulea validator) fără modificarea codului existent, validând în practică Principiul Open/Closed;
- respectarea principiilor SOLID și a bunelor practici de inginerie software: cod curat, tranzacționalitate corectă, testabilitate, separarea responsabilităților.

Rezultatul final este o aplicație publicată pe GitHub la adresa github.com/mihail-chircu/TMPPP, care poate fi rulată local cu instrucțiunile din fișierul README și care conține toate cele cincisprezece pattern-uri documentate în prezentul raport.

2 PROIECTAREA APLICAȚIEI

2.1 Arhitectura generală Kinder

Arhitectura Kinder urmează paradigma MVC impusă de Laravel, cu câteva niveluri suplimentare introduse pentru a găzdui logica de business. Controller-ele (directorul `app/Http/Controllers`) sunt limitate strict la responsabilitățile HTTP: primirea request-ului, validarea input-ului și delegarea procesării către layer-ul de servicii. Modelele Eloquent (`app/Models`) reprezintă entitățile de domeniu — Product, Category, Order, OrderItem, Cart, CartItem, Discount, User — și expun relațiile dintre ele.

Între controller și model trăiește layer-ul de servicii (`app/Services`), organizat pe subdomenii: Checkout, Order, Cart, Catalog, Pricing, Payment, Shipping, Validation, Export, OrderState. Fiecare subdomeniu conține pattern-urile specifice problemei sale. De exemplu, Pricing conține Decorator-ul cu BasePrice și cei trei decoratori, iar Shipping conține Factory Method-ul cu cele trei metode și cele trei calculatoare.

Subsistemele transversale sunt organizate separat: Notifications (Abstract Factory pentru notificări multi-canal), Repositories (Proxy pentru caching), Events și Listeners (Observer pentru reacții post-comandă), Contracts (interfețele împărtășite de mai multe pattern-uri). AppServiceProvider joacă rolul central de „wire-up”: asociază interfețele la implementările concrete prin container-ul de dependențe Laravel. Clienții primesc astfel prin DI versiunea corectă a obiectului, fără să o instantieze explicit.

În continuare, fiecare pattern este prezentat într-o secțiune proprie cu următoarea structură: descrierea problemei rezolvate în context Kinder, diagrama UML a componentelor, descrierea rolurilor fiecărei clase participante și explicația modului în care pattern-ul se integrează în fluxul global al aplicației.

2.2 Pattern-uri creaționale

Pattern-urile creaționale abstractizează procesul de instanțiere a obiectelor. În proiectul Kinder au fost implementate patru pattern-uri din această categorie, acoperind construcția comenzilor (Builder), clonarea produselor (Prototype), selectarea calculatorului de shipping (Factory Method) și producerea notificărilor pe diverse canale (Abstract Factory).

2.2.1 Pattern-ul Builder

Entitatea Order din aplicația Kinder are peste cincisprezece câmpuri obligatorii (numărul comenzii, utilizatorul, numele și emailul clientului, telefonul, adresa completă de livrare, orașul, metoda de plată, metoda de livrare, costul de livrare, subtotalul, totalul, statusul, notele clientului și lista de produse comandate), iar construirea ei nu este o simplă inițializare de atribute. Pe lângă asignarea câmpurilor, procesul presupune logică colaterală: decrementarea stocului fiecărui produs comandat, incrementarea

contorului de vânzări, aplicarea decoratorilor de preț (ambalaj cadou, taxe) și persistarea atomică a comenzii împreună cu liniile sale de produs într-o tranzacție de bază de date.

Un constructor cu cincisprezece parametri sau o serie de metode overload ar fi fragile și dificil de testat. Pattern-ul Builder separă construcția complexă a obiectului de reprezentarea sa finală, permițând asamblarea pas cu pas a unei comenzii prin apeluri succesive la o interfață fluentă, iar materializarea în baza de date este amânată până la apelul final build().

Diagrama UML a componentelor pattern-ului Builder este prezentată în figura 2.1.

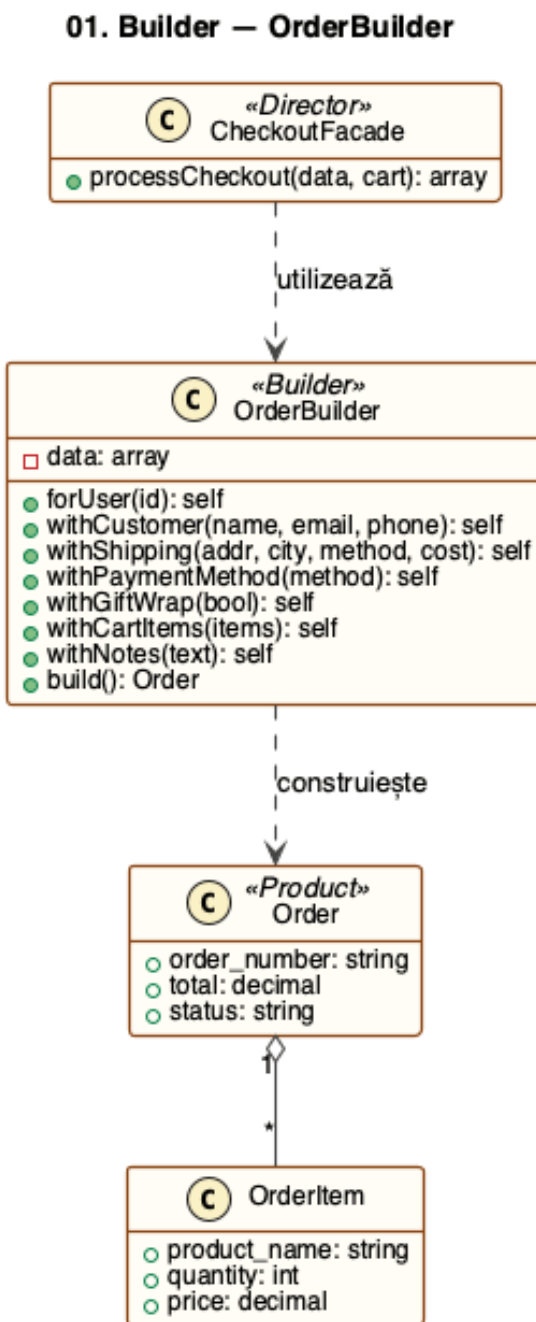


Figura 2.1 – Diagrama UML a pattern-ului Builder

În implementarea concretă, rolul de Builder este jucat de clasa `OrderBuilder`, care expune metodele `forUser`, `withCustomer`, `withShipping`, `withPaymentMethod`, `withShippingMethod`, `withGiftWrap`, `withCartItems` și `withNotes`. Rolul de Director este asumat de `CheckoutFacade`, ce orchestrează apelurile succesive pe builder în cadrul unei singure tranzacții de checkout. Rolul de Product este îndeplinit de modelul Eloquent `Order`, care împreună cu entitățile `OrderItem` asociate formează rezultatul final al construcției.

La finalizarea unei comenzi, controller-ul de checkout invocă `CheckoutFacade::processCheckout`, care la rândul său populează `OrderBuilder`-ul cu toate datele validate. Construcția se finalizează prin `DB::transaction`, garantând că fie toate side-effect-urile se aplică (crearea `Order`-ului, crearea `OrderItem`-urilor, decrementarea stocurilor), fie niciunul — consistența datelor este astfel preservată chiar și în cazul unor erori de rețea sau concurență.

2.2.2 Pattern-ul Prototype

Magazinul Kinder comercializează un catalog de peste treizeci de jucării, iar administratorul trebuie adesea să creeze variante ale unui produs existent — de exemplu, aceeași mașinuță cu telecomandă în altă culoare, cu preț diferit sau cu un set de accesorii alternativ. A introduce manual douăzeci de câmpuri pentru fiecare variantă ar fi consumator de timp și generator de erori, iar cuplarea codului la clasa concretă `Product` nu este dorită, pentru că noile variante pot include colecții înrudite (imagini, tag-uri, reduceri).

Pattern-ul Prototype rezolvă această problemă declarând o interfață minimală cu metoda duplicate, pe care orice clasă capabilă de clonare profundă o implementează. Clientul (controller-ul administrativ) cere o copie a instanței curente fără să îi cunoască tipul concret, iar prototype-ul știe singur cum să se copieze corect.

Diagrama UML a componentelor pattern-ului Prototype este prezentată în figura 2.2.

02. Prototype — Product::duplicate()

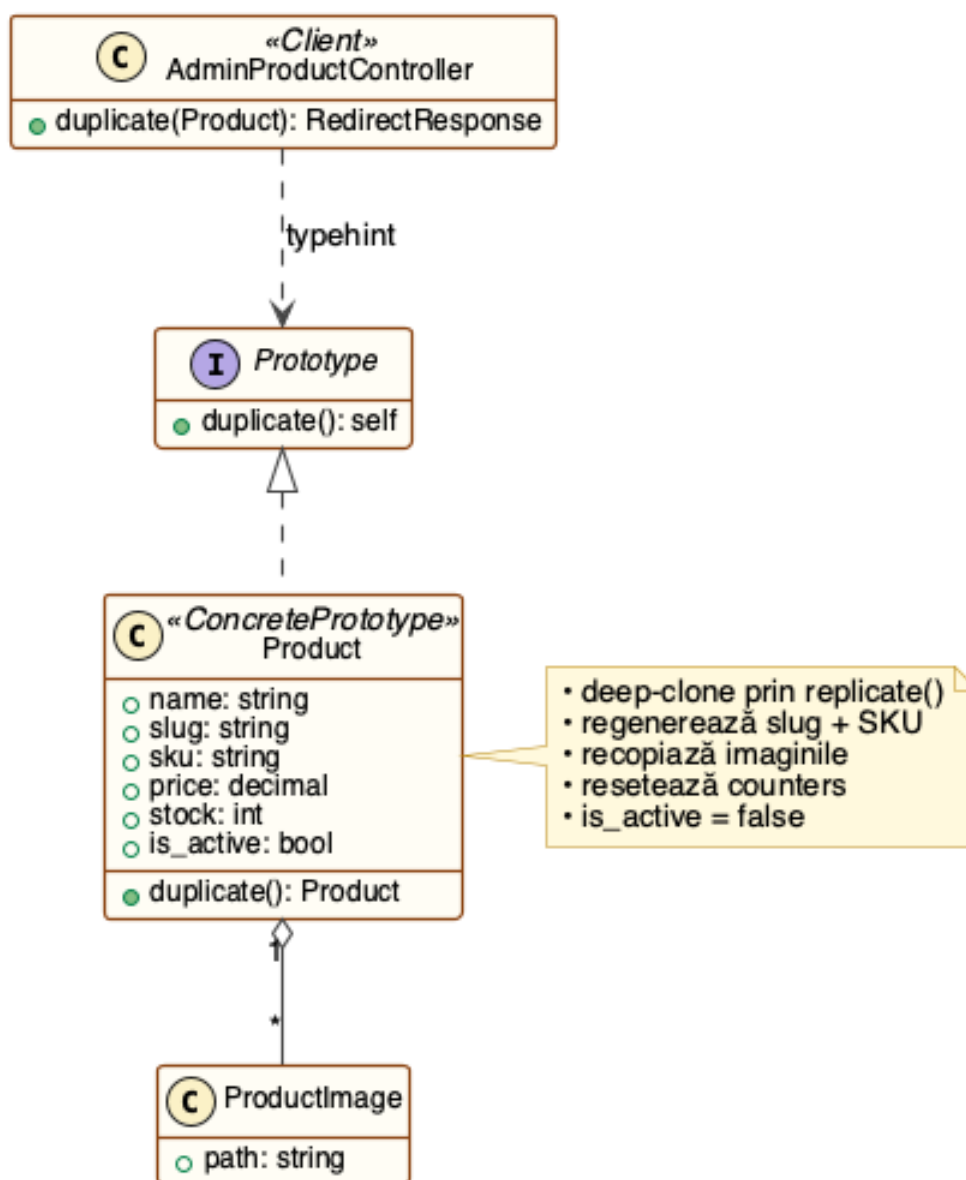


Figura 2.2 – Diagrama UML a pattern-ului Prototype

Interfața `Prototype` declară metoda `duplicate(): self`. Rolul de Concrete Prototype este îndeplinit de modelul `Product`, care implementează `duplicate` prin apelul Eloquent `replicate()`, urmat de regenerarea slug-ului și a SKU-ului (pentru a asigura unicitatea), recopierea relației `ProductImage` și resetarea contoarelor `views_count` și `sales_count`. Noul produs primește flag-ul `is_active=false`, astfel încât să nu apară în catalogul public până când administratorul îi ajustează câmpurile modificate.

În panoul administrativ, fiecare rând din listarea produselor conține un buton „Duplicate” care apelează `AdminProductController::duplicate`. Acesta invocă `$product->duplicate()` — clientul nu știe dacă primește un `Product` sau o subclasă, iar responsabilitatea copierii profunde revine în întregime prototype-ului. Astfel, adăugarea unui câmp nou în model nu cere modificări în controller.

2.2.3 Pattern-ul Factory Method

Aplicația oferă trei metode de livrare (Standard, Express, Gratuită), fiecare cu o formulă de preț radical diferită. Livrarea standard se calculează ca 35 MDL plus o taxă de 3 MDL per produs suplimentar, plus 15 MDL suprataxă pentru livrări în afara Chișinăului și un discount de 5 MDL pentru comenzile de peste 300 MDL. Livrarea expres adaugă o primă de asigurare de 1% capped la 40 MDL și o suprataxă de weekend. Livrarea gratuită este disponibilă doar peste pragul de 500 MDL.

A codifica aceste reguli într-un if-else monolitic ar cupla orice modificare a unei metode la întregul serviciu de shipping. Pattern-ul Factory Method izolează logica de creare a calculatorului potrivit: alegerea metodei de livrare de către utilizator determină implicit ce calculator se instanțiază, iar codul client aplică pur și simplu calculate() fără să cunoască varianta concretă.

Diagrama UML a componentelor pattern-ului Factory Method este prezentată în figura 2.3.

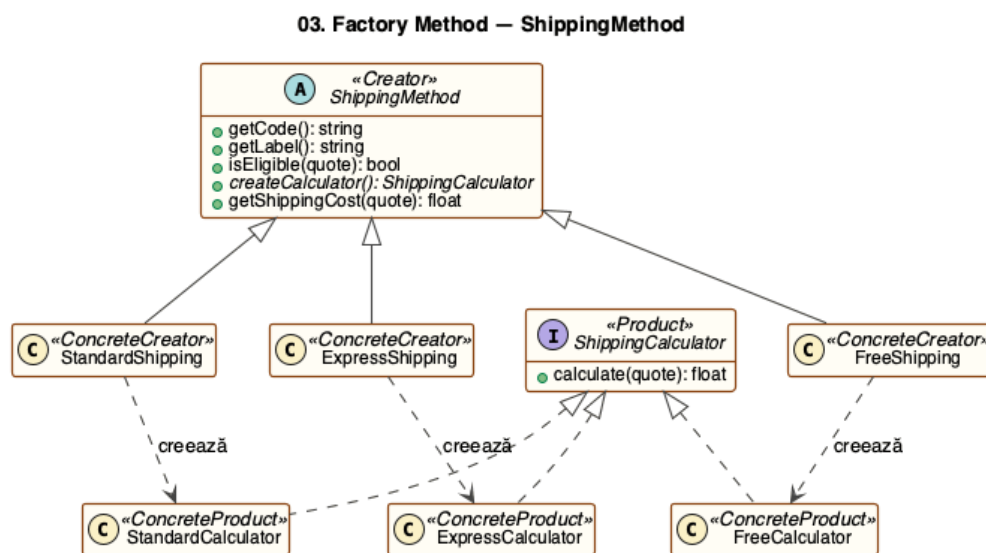


Figura 2.3 – Diagrama UML a pattern-ului Factory Method

Clasa abstractă **ShippingMethod** joacă rolul de Creator, definind metoda fabrică `createCalculator()`: **ShippingCalculator**. Concrete Creator-ele sunt **StandardShipping**, **ExpressShipping** și **FreeShipping**. Interfața **ShippingCalculator** reprezintă Product-ul, implementat de cele trei Concrete Products: **StandardCalculator**, **ExpressCalculator** și **FreeCalculator**. Fiecare metodă de shipping întoarce calculatorul propriu, iar logica specifică (formula de preț) este încapsulată în calculatorul concret, nu în metoda de shipping.

CheckoutFacade construiește un obiect **ShippingQuote** (total, număr de produse, orașul de livrare) și îl transmite la **ShippingService**. Acesta selectează **ShippingMethod**-ul cerut, verifică dacă este eligibil prin `isEligible(quote)` (doar Free are o restricție de prag) și apelează `getShippingCost`, care intern folosește metoda fabrică. Adăugarea unei metode noi (de exemplu, livrare prin curier rapid) cere o clasă nouă pentru metoda și una pentru calculator, fără a atinge codul existent.

2.2.4 Pattern-ul Abstract Factory

Aplicația Kinder trimite trei tipuri de notificări legate de comandă — confirmare plasare, update de status și expediere — pe trei canale distincte: Email, SMS și notificări Push în aplicația mobilă. Fiecare canal are constrângeri proprii: emailul poate fi narativ și include lista detaliată a produselor, SMS-ul trebuie să respecte limita de 160 de caractere cu un mesaj concis, iar Push-ul trebuie să aibă un titlu scurt plus un body minimal cu deep-link. Mai mult, cele trei tipuri de notificări diferă structural: confirmarea include un sumar al produselor și totalul, update-ul de status poartă statusul vechi și cel nou, iar notificarea de expediere conține codul de tracking și data estimată.

Pattern-ul Abstract Factory produce familii de produse legate între ele: o alegere unică (canalul) determină o familie completă și consistentă de notificări. Codul client primește un factory și îl apelează fără să cunoască variantele concrete, iar extinderea sistemului cu un nou canal este o singură clasă nouă.

Diagrama UML a componentelor pattern-ului Abstract Factory este prezentată în figura 2.4.

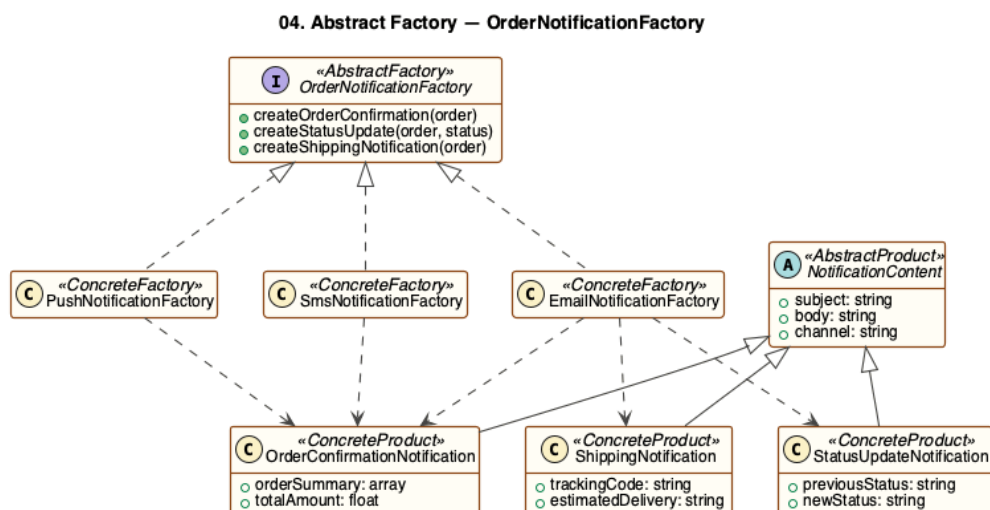


Figura 2.4 – Diagrama UML a pattern-ului Abstract Factory

Interfața OrderNotificationFactory joacă rolul de Abstract Factory, declarând trei metode create: createOrderConfirmation, createStatusUpdate și createShippingNotification. Concrete Factory-urile sunt EmailNotificationFactory, SmsNotificationFactory și PushNotificationFactory, fiecare producând variantele tunate pentru canalul propriu. Clasa abstractă NotificationContent reprezintă Abstract Product, moștenită de trei Concrete Products: OrderConfirmationNotification (cu orderSummary și totalAmount), StatusUpdateNotification (cu previousStatus și newStatus) și ShippingNotification (cu trackingCode și estimatedDelivery).

AppServiceProvider înregistrează canalul implicit (Email) pe interfață prin container-ul de dependențe Laravel. OrderNotificationService primește factory-ul prin injecție și îl folosește fără cunoaștere a canalului: schimbarea implementării se face într-o singură linie, fără a atinge serviciul. Adăugarea canalului Push, realizată în proiect ca o îmbunătățire recentă, a necesitat doar crearea clasei

PushNotificationFactory — nicio altă parte a codului nu a fost modificată, validând în practică Principiul Open/Closed.

2.3 Pattern-uri structurale

Pattern-urile structurale descriu modul în care clase și obiecte se compun pentru a forma structuri mai mari, păstrând în același timp flexibilitatea. În Kinder au fost aplicate cinci: Facade pentru orchestrarea fluxului de checkout, Decorator pentru compunerea dinamică a prețului final, Adapter pentru uniformizarea API-urilor gateway-urilor de plată, Proxy pentru caching-ul transparent al interogărilor către pagina de start și Composite pentru modelarea arborelui de categorii.

2.3.1 Pattern-ul Facade

Procesul de checkout implică cel puțin șase subsisteme independente: validarea datelor și a coșului (Chain of Responsibility), calculul costului de livrare (Factory Method), asamblarea comenzii în baza de date (Builder), procesarea plății prin gateway-ul selectat (Adapter), curățarea istoricului de comenzi din sesiune (Command Invoker) și declanșarea reacțiilor post-comandă (Observer). Un controller care ar orchestra direct toate aceste subsisteme ar fi cuplat la șase clase concrete, dificil de testat și de întreținut.

Pattern-ul Facade introduce o fațadă care expune o interfață simplă pentru utilizarea unui subsistem complex. Clientul — în cazul de față CheckoutController — apelează o singură metodă a fațadei, iar fațada se ocupă de orchestrarea intern.

Diagrama UML a componentelor pattern-ului Facade este prezentată în figura 2.5.

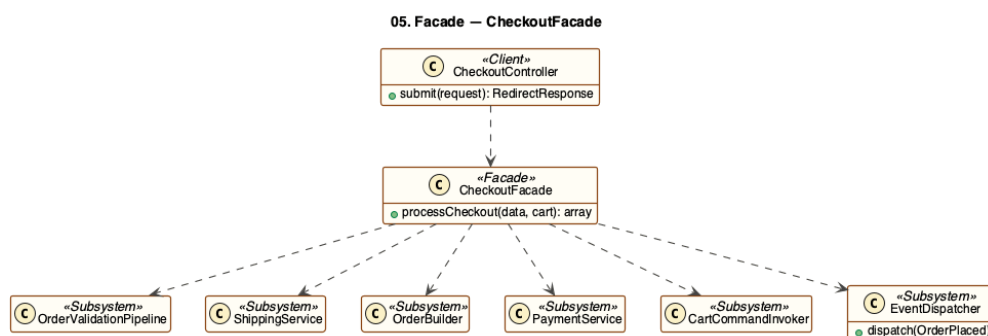


Figura 2.5 – Diagrama UML a pattern-ului Facade

Clasa CheckoutFacade joacă rolul central, expunând metoda processCheckout(data, cart). Ea primește prin constructor referințe la subsistemele ascunse: OrderValidationPipeline, ShippingService, OrderBuilder, PaymentService, CartCommandInvoker și event dispatcher-ul Laravel. Client-ul este CheckoutController, care după validarea form-ului delegă integral procesul la fațadă.

Controller-ul validează câmpurile HTTP ale request-ului și apelează processCheckout. Fațada rulează validările prin Chain, construiește un ShippingQuote și calculează costul, invocă OrderBuilder pentru asamblarea atomică a comenzii, curăță coșul și istoricul acțiunilor, inițializează plata prin adaptorul

corespunzător și dispatch-uiște evenimentul OrderPlaced pentru observatori. Toate acestea sunt invizibile pentru controller, care rămâne astfel simplu și concentrat pe responsabilitatea sa HTTP.

2.3.2 Pattern-ul Decorator

Prețul final al unei comenzi este compus din mai multe modificatori care pot fi aplicați în orice combinație și ordine: subtotalul de bază, o taxă fixă pentru ambalaj cadou de 25 MDL (opțională), un discount procentual (condiționat de prag), taxa pe valoare adăugată. În viitor pot apărea modificatori suplimentari — cupon de fidelitate, taxă de ambalaj ecologic, garanție extinsă. A codifica toate combinațiile prin if-uri în clasa Order ar duce la explozia combinatorică a condițiilor.

Pattern-ul Decorator permite compunerea dinamică a responsabilităților prin împachetarea succesivă a unui obiect într-un lanț de decoratori care adaugă fiecare câte o contribuție. Rezultatul final se obține prin traversarea în sens invers a lanțului, fiecare decorator adăugând valoarea proprie la cea a componentei împachetate.

Diagrama UML a componentelor pattern-ului Decorator este prezentată în figura 2.6.

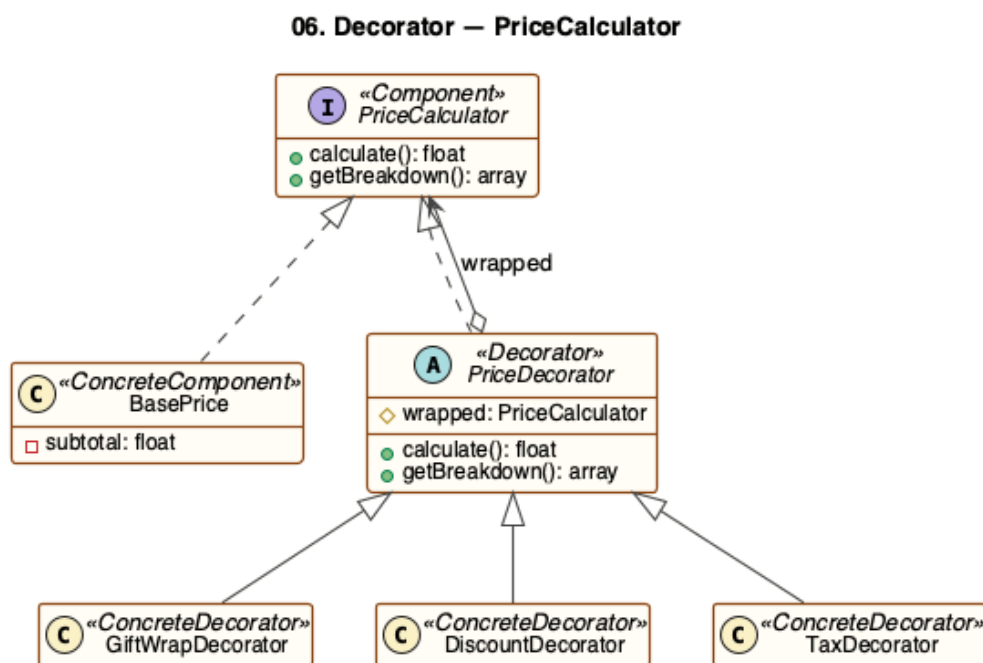


Figura 2.6 – Diagrama UML a pattern-ului Decorator

Interfața PriceCalculator joacă rolul de Component și declară `calculate(): float` plus `getBreakdown(): array`. Concrete Component-ul este BasePrice, care întoarce pur și simplu subtotalul transmis. Clasa abstractă PriceDecorator definește Decorator-ul, ținând o referință la un alt PriceCalculator împachetat. Concrete Decorators sunt GiftWrapDecorator (adaugă o taxă fixă), DiscountDecorator (aplică un procent) și TaxDecorator (adaugă TVA). Fiecare decorator forwardează apelul la wrapped și adaugă propria contribuție.

La construcția comenzii, `OrderBuilder::buildPrice` construiește lanțul de decoratori în funcție de opțiunile alese de client. De exemplu, dacă utilizatorul bifează „ambalaj cadou”, codul creează `new GiftWrapDecorator(new BasePrice($subtotal))`. Metoda `getBreakdown` agregă contribuțiile fiecărui decorator ca linii separate pe chitanța finală. Adăugarea unui modificador nou nu cere modificări în `BasePrice` și nici în apelanți — doar o clasă nouă care extinde `PriceDecorator`.

2.3.3 Pattern-ul Adapter

Aplicația trebuie să proceseze plăți prin mai multe canale cu API-uri fundamental diferite. Gateway-ul Stripe așteaptă sumele în cenți (nu în MDL), currency-ul în minuscule, carduri tokenizate și returnează un obiect cu câmpurile `id`, `status`, `amount_captured` și `card.last4`. API-ul bancar local folosește coduri IBAN și SWIFT, are o mașină de stări proprie cu tranzițiile `PENDING_CONFIRMATION` → `SETTLED` sau `REJECTED` și returnează date de tip ISO-8601. Plata la livrare nu are deloc API — doar trebuie marcată ca „neplătită încă”.

Restul aplicației nu trebuie să cunoască aceste diferențe. Pattern-ul Adapter rezolvă incompatibilitatea între interfețe, traducând API-ul client într-o interfață unificată cu care `PaymentService` lucrează indiferent de canalul real.

Diagrama UML a componentelor pattern-ului Adapter este prezentată în figura 2.7.

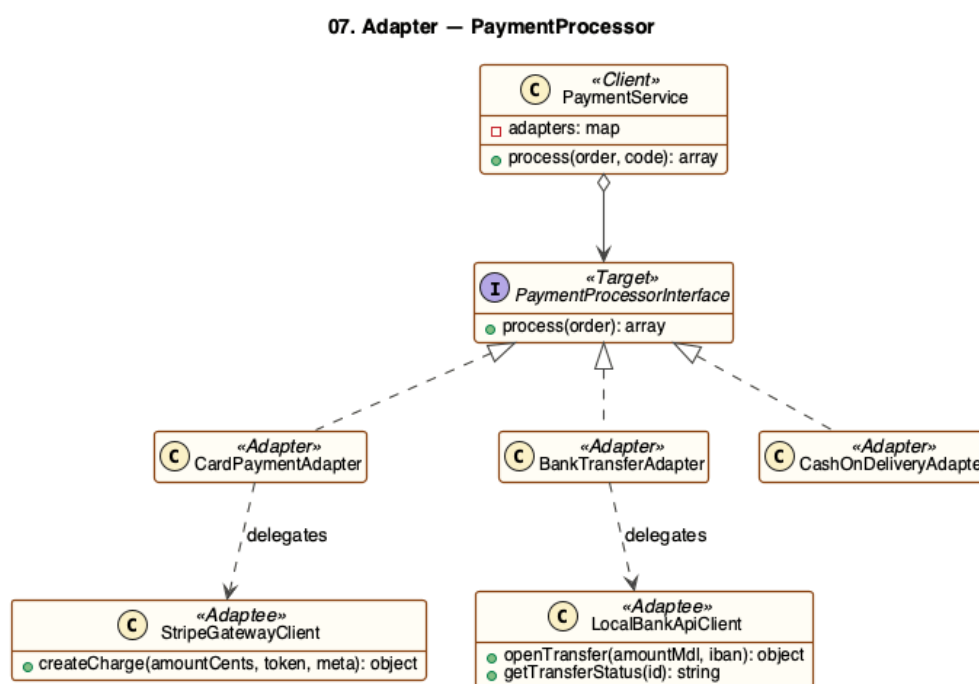


Figura 2.7 – Diagrama UML a pattern-ului Adapter

Interfața `PaymentProcessorInterface` joacă rolul de Target, expunând metoda `process(order): array` cu un shape unificat `{success, transaction_id, message}`. Adaptees sunt clienții externi `StripeGatewayClient` și `LocalBankApiClient`. Adapters sunt `CardPaymentAdapter` (convertește suma MDL în cenți, tokenizează

cardul, traduce răspunsul Stripe), BankTransferAdapter (deschide un transfer, interpretează mașina de stări, returnează URL-ul de instrucțiuni) și CashOnDeliveryAdapter (nu are adaptee — marchează direct comanda ca „plată la livrare”). Client-ul este PaymentService, care deține un dicționar code → adapter și apelează doar interfața unificată.

La pasul de plată, CheckoutFacade cere PaymentService::process(\$order, \$code). Service-ul caută adaptorul corespunzător codului și îi apelează process. Exemplu: pentru cod „card”, CardPaymentAdapter transformă totalul 617 MDL în 61700 cenți, generează un token, apelează StripeGatewayClient::createCharge și mapează răspunsul. Aplicația nu „știe” și nu are nevoie să „știe” că gateway-ul vorbește într-o altă limbă decât domeniul nostru.

2.3.4 Pattern-ul Proxy

Pagina de start a magazinului Kinder afișează patru secțiuni care fac interogări costisitoare în baza de date: produse „featured”, „new arrivals”, „on sale” și „most popular”. Fiecare vizită anonimă repetă aceste interogări identic, consumând inutil resurse ale bazei de date și încetinind timpul de încărcare a paginii. Soluția naturală este caching-ul, dar introducerea lui direct în controller ar amesteca responsabilitățile: controller-ul ar trebui să știe despre cache, chei, TTL și strategii de expirare.

Pattern-ul Proxy oferă un substituent pentru un obiect real, controlând accesul la acesta. În cazul nostru, un Caching Proxy interceptează apelurile la repository-ul de produse și cache-uiește rezultatele transparent, fără ca vreo parte a codului client să fie afectată.

Diagrama UML a componentelor pattern-ului Proxy este prezentată în figura 2.8.

08. Proxy — CachingProductProxy

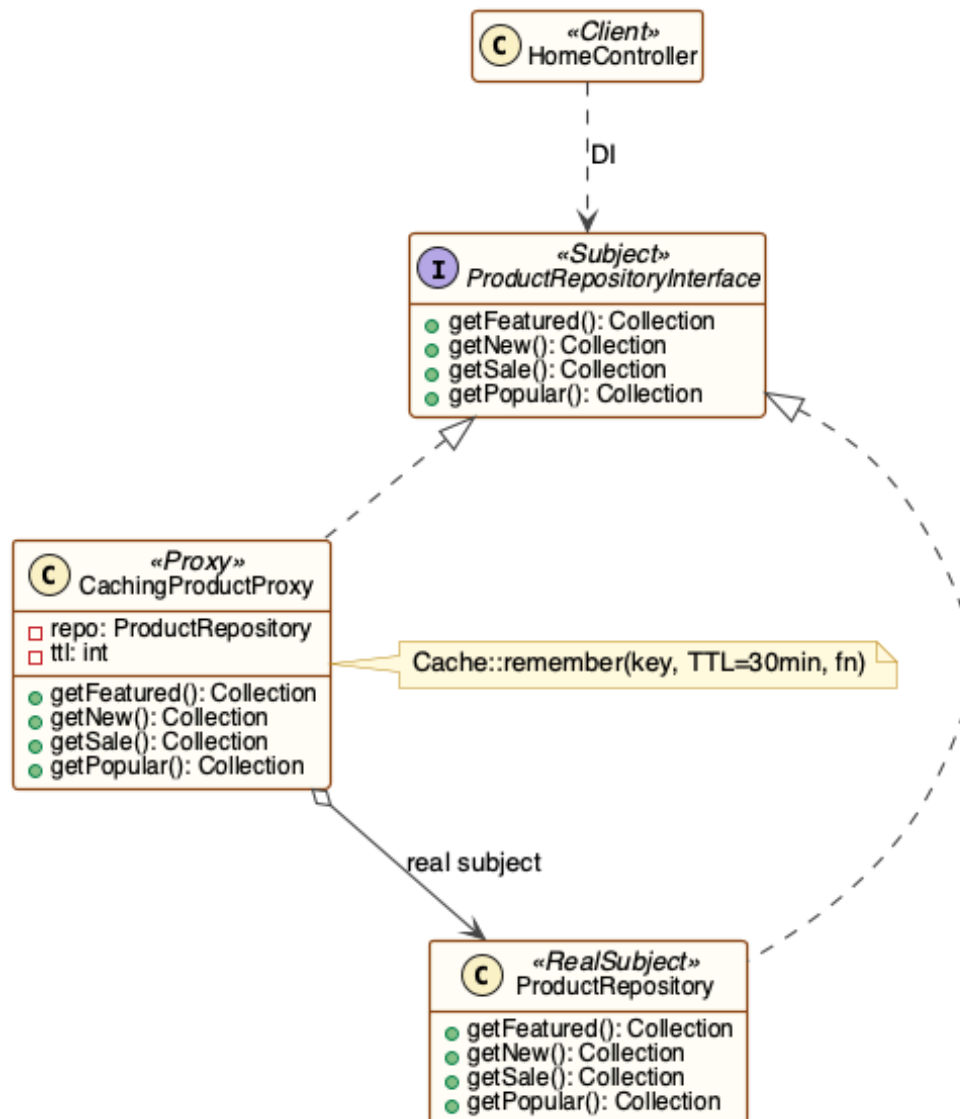


Figura 2.8 – Diagrama UML a pattern-ului Proxy

Interfața `ProductRepositoryInterface` joacă rolul de Subject, cu metodele `getFeatured`, `getNew`, `getSale` și `getPopular`. Real Subject-ul este `ProductRepository`, care execută query-urile Eloquent reale pe baza de date. Proxy-ul este `CachingProductProxy`, care împachetează real subject-ul și înfășoară fiecare metodă într-un apel `Cache::remember` cu TTL de 30 de minute. Client-ul este `HomeController`, care primește interfața prin injecție de dependență.

`AppServiceProvider` face bind pe `ProductRepositoryInterface` către `CachingProductProxy`, astfel încât `HomeController` primește direct proxy-ul. Primul apel la `getFeatured` atinge baza de date și populează cache-ul; apelurile următoare din fereastra de 30 de minute returnează rezultatul din cache fără a atinge DB-ul. Un avantaj suplimentar este flexibilitatea: schimbarea provider-ului de cache (în Redis, Memcached sau alt driver) se face într-o singură linie de configurație, fără modificări în controller.

2.3.5 Pattern-ul Composite

Catalogul Kinder este organizat pe categorii ierarhice: categoria „Jucării din pluș” conține subcategoriile „Ursuleți” și „Jucării moi muzicale”, în timp ce „Mașini cu telecomandă” conține „Mașini off-road” și „Drone”. Adâncimea arborelui poate varia în funcție de extinderea catalogului. Codul client (sidebar-ul de navigare, pagina administrativă) are nevoie să afișeze pentru fiecare categorie numărul total de produse, care trebuie să includă recursiv și subcategoriile — dar sidebar-ul nu trebuie să se complice cu distincția între o frunză și un compozit.

Pattern-ul Composite unifică tratamentul obiectelor individuale și al grupurilor de obiecte printr-o interfață comună. Clientul apelează aceeași metodă pe orice nod al arborelui, iar rezultatul este calculat recursiv, ascunzând complexitatea traversării.

Diagrama UML a componentelor pattern-ului Composite este prezentată în figura 2.9.

09. Composite — Category

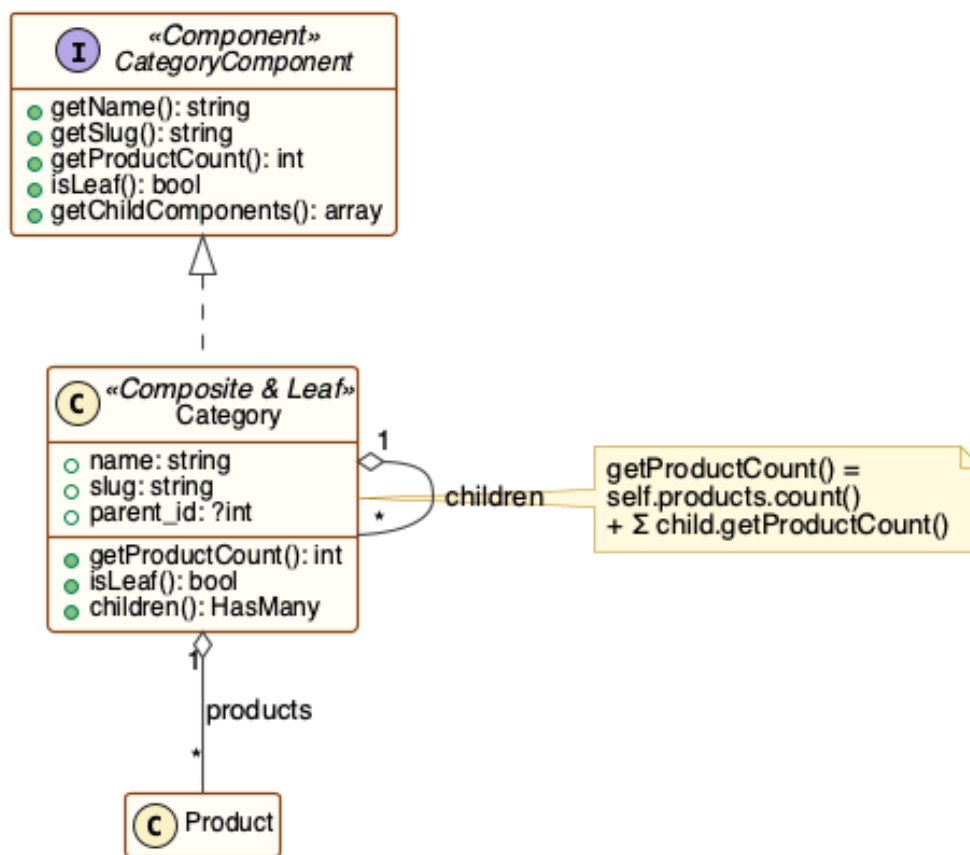


Figura 2.9 – Diagrama UML a pattern-ului Composite

Interfața `CategoryComponent` joacă rolul de `Component`, declarând metodele `getName`, `getSlug`, `getProductCount`, `isLeaf` și `getChildComponents`. Atât `Leaf`, cât și `Composite` sunt reprezentate de aceeași clasă concretă `Category`: o instanță fără copii se comportă ca frunză, iar una cu copii se comportă ca un

compozit. Implementarea `getProductCount` este recursivă — numără propriile produse asociate, apoi adună la acestea contribuția fiecărui copil activ.

Arborele de categorii este persistat în baza de date prin relația self-referințială `parent_id → children`. Adăugarea unei noi subcategorii nu cere cod nou — doar un rând în DB. Sidebar-ul catalog iterează recursiv prin arbore pornind de la categoriile rădăcină și apelează `getProductCount` pe fiecare nod; indiferent de adâncime, comportamentul este uniform.

2.4 Pattern-uri comportamentale

Pattern-urile comportamentale vizează distribuirea responsabilităților și fluxul de control între obiecte. În Kinder au fost integrate șase: Strategy pentru sortare interschimbabilă a catalogului, State pentru mașina de stări a comenzii, Command pentru reversibilitatea acțiunilor pe coș, Chain of Responsibility pentru validarea înlănțuită a checkout-ului, Template Method pentru exportul comenzilor în formate multiple și Observer pentru reacțiile post-comandă.

2.4.1 Pattern-ul Strategy

Catalogul de produse oferă mai multe moduri de sortare: după data adăugării, după popularitate, după preț crescător sau descrescător. În viitor se pot adăuga strategii noi — „cele mai bine cotate de clienți”, „cele mai recent reduse”. Fiecare sortare are o logică proprie care se aplică pe query-ul Eloquent, iar o implementare cu switch sau if-else ar necesita modificarea controller-ului la fiecare variantă nouă.

Pattern-ul Strategy definește o familie de algoritmi, îi încapsulează pe fiecare într-o clasă separată și îi face interschimbabili. Clientul alege strategia la runtime în funcție de input (query string) și o aplică pe context, fără a cunoaște detaliile implementării.

Diagrama UML a componentelor pattern-ului Strategy este prezentată în figura 2.10.

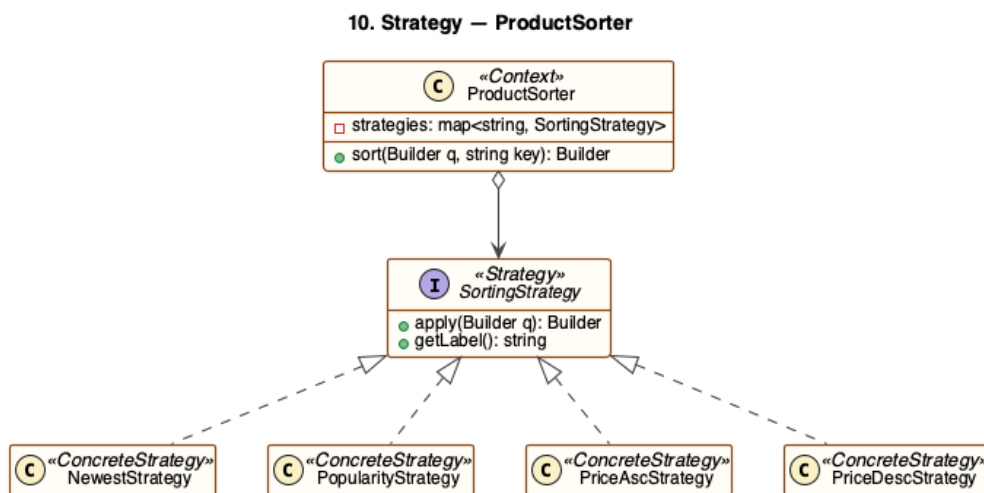


Figura 2.10 – Diagrama UML a pattern-ului Strategy

Interfața `SortingStrategy` joacă rolul de `Strategy` cu metoda `apply(Builder q): Builder`. Concrete `Strategies` sunt `NewestStrategy`, `PopularityStrategy`, `PriceAscStrategy` și `PriceDescStrategy`, fiecare aplicând propriul `orderBy` pe `query`. Context-ul este `ProductSorter`, care deține un dicționar cheie → strategie și rezolvă cheia primită din `query string`.

`CatalogController` primește parametrul `sort` din URL-ul de catalog și îl transmite la `ProductSorter`, care aplică strategia corespunzătoare pe `query-ul Eloquent`, iar rezultatul este paginat și trimis la `view`. O strategie nouă înseamnă o singură clasă care implementează `SortingStrategy` — fără modificări în `controller`.

2.4.2 Pattern-ul State

O comandă `Kinder` poate fi în una din cinci stări: în așteptare (`pending`), în procesare (`processing`), expediată (`shipped`), livrată (`delivered`) sau anulată (`cancelled`). Tranzițiile între stări sunt reglementate: din „`pending`” se poate trece în „`processing`” sau „`cancelled`”, dar nu direct în „`delivered`”; din „`shipped`” nu se mai poate anula. Codificarea acestor reguli printr-un mare `if-else` pe câmpul `status` ar fi greu de întreținut și supus erorilor, mai ales când adăugăm acțiuni specifice per tranziție (trimiterea notificării de expediere, setarea codului de tracking).

Pattern-ul `State` permite unui obiect să își modifice comportamentul în funcție de starea sa internă, ca și cum și-ar schimba clasa. Fiecare stare este reprezentată de o clasă separată care cunoaște regulile de tranziție și acțiunile permise din acea stare.

Diagrama UML a componentelor pattern-ului `State` este prezentată în figura 2.11.

11. State — OrderState

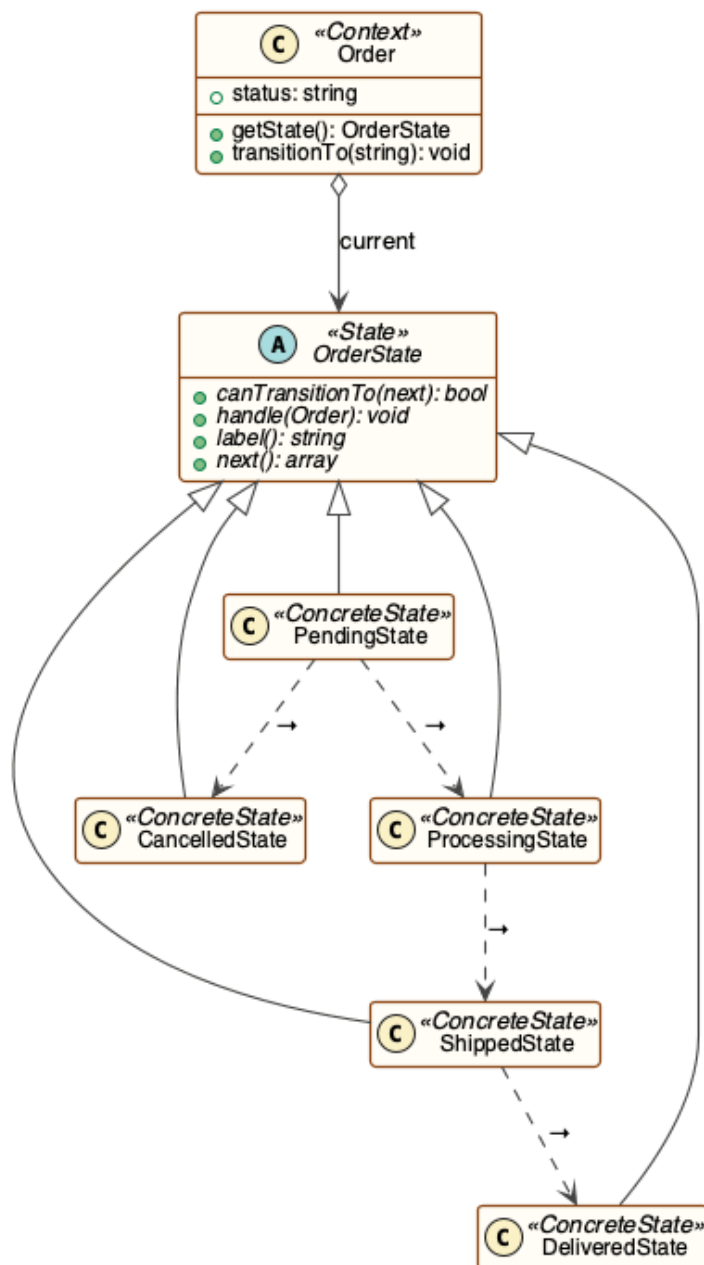


Figura 2.11 – Diagrama UML a pattern-ului State

Clasa abstractă `OrderState` joacă rolul de State, declarând metodele `canTransitionTo(next): bool`, `handle(order): void`, `label(): string` și `next(): array`. Concrete States sunt `PendingState`, `ProcessingState`, `ShippedState`, `DeliveredState` și `CancelledState`. Context-ul este modelul `Order` însuși, care ține un câmp `status` și oferă metoda `getState()` ce returnează instanța State-ului corespunzător.

Când administratorul schimbă statusul unei comenzi din panoul admin, controller-ul cere starea curentă, întreabă `canTransitionTo` pentru starea dorită, execută `handle` (care poate declanșa side-effect-uri — de exemplu `ShippedState` trimite notificarea de expediere și generează codul de tracking) și persistă noul status. Adăugarea unei stări noi, cum ar fi „returned”, este o clasă nouă — state-urile existente nu sunt atinse.

2.4.3 Pattern-ul Command

Coșul de cumpărături acceptă trei acțiuni: adăugarea unui produs, modificarea cantității și eliminarea unui produs. Utilizatorii așteaptă reversibilitate — un buton „Anulează ultima operație” care să restabilească starea precedentă, utilizat și între request-uri HTTP succesive. Implementarea naivă cu metode instantanee pe CartService nu păstrează istoric; un client care a eliminat accidental un produs nu îl mai poate recupera.

Pattern-ul Command transformă fiecare cerere într-un obiect de sine stătător, conținând toate informațiile necesare pentru a executa cererea sau a o inversa. Istoricul devine posibil, undo-ul devine trivial, iar serializarea permite păstrarea comenzilor între request-uri.

Diagrama UML a componentelor pattern-ului Command este prezentată în figura 2.12.

12. Command – CartCommand (cu Undo)

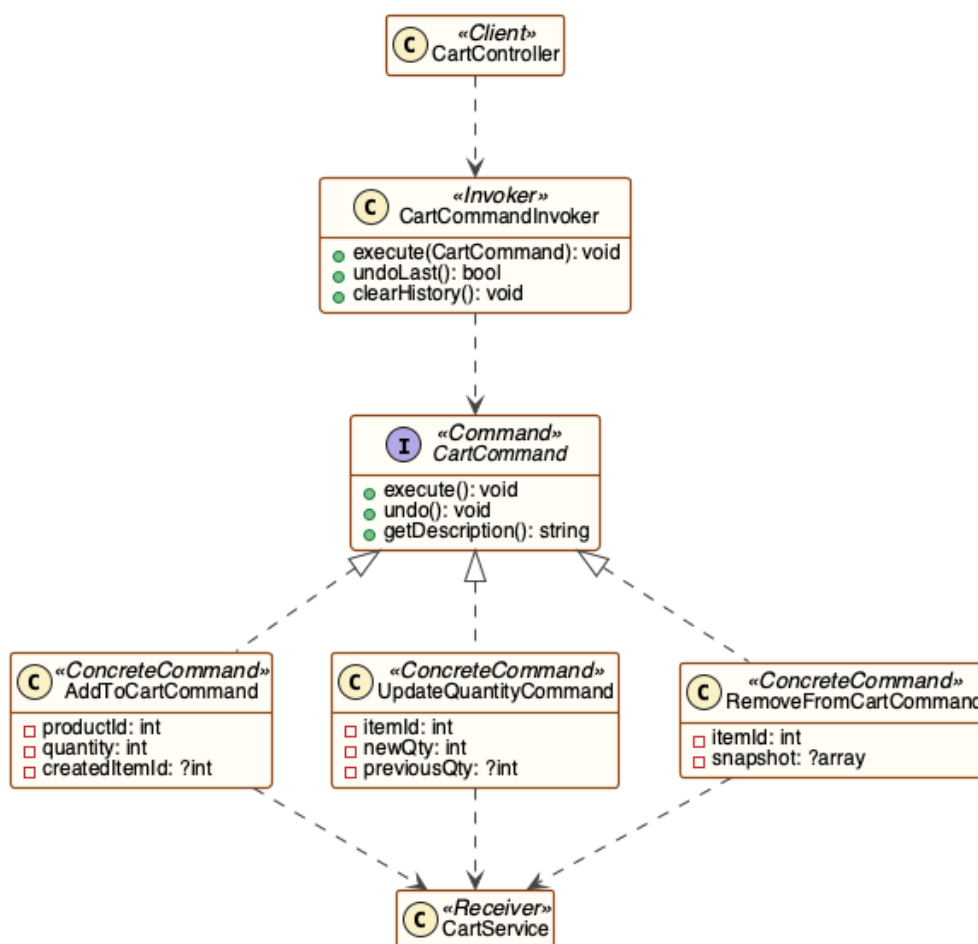


Figura 2.12 – Diagrama UML a pattern-ului Command

Interfața *CartCommand* joacă rolul de Command cu metodele `execute`, `undo` și `getDescription`. Concrete Commands sunt *AddToCartCommand* (păstrează id-ul item-ului creat pentru a-l putea șterge la undo), *UpdateQuantityCommand* (snapshot-uește cantitatea precedentă) și *RemoveFromCartCommand* (snapshot-uește întregul rând înainte de ștergere, pentru a-l putea recrea). Invoker-ul este

CartCommandInvoker, care execută comenzile și le serializează în sesiunea Laravel. Receiver-ul este CartService.

La fiecare modificare a coșului, CartController construiește Command-ul potrivit și îl execută prin invoker, care îl păstrează în stiva din sesiune. În view, un banner afișează getDescription al ultimei comenzi și un buton „Anulează”; click-ul trimite un POST la /cart/undo, invoker-ul dez-serializează ultima comandă și apelează undo. După finalizarea cu succes a unui checkout, CheckoutFacade apelează clearHistory — comenzile nu mai pot fi anulate odată consumate.

2.4.4 Pattern-ul Chain of Responsibility

Înainte ca o comandă să fie plasată, trebuie verificate patru aspecte independente: disponibilitatea stocului pentru fiecare produs, corectitudinea prețurilor (discount-urile active pot expira între momentul adăugării în coș și plasarea comenzii), atingerea pragului minim al comenzii și validitatea adresei de livrare. Fiecare validare are date proprii și mesaje de eroare proprii. Un singur validator monolitic ar deveni rapid greu de extins, iar adăugarea unui cec suplimentar (de exemplu, verificarea unui cod promoțional) ar necesita modificarea codului existent.

Pattern-ul Chain of Responsibility permite lanțuirea mai multor handler-i care procesează o cerere succesiv. Fiecare handler decide fie să trateze cererea, fie să o paseze mai departe. Pentru un validator, tratarea înseamnă oprirea lanțului cu un rezultat de eroare; pasarea înseamnă delegarea la următorul.

Diagrama UML a componentelor pattern-ului Chain of Responsibility este prezentată în figura 2.13.

13. Chain of Responsibility — OrderValidationPipeline

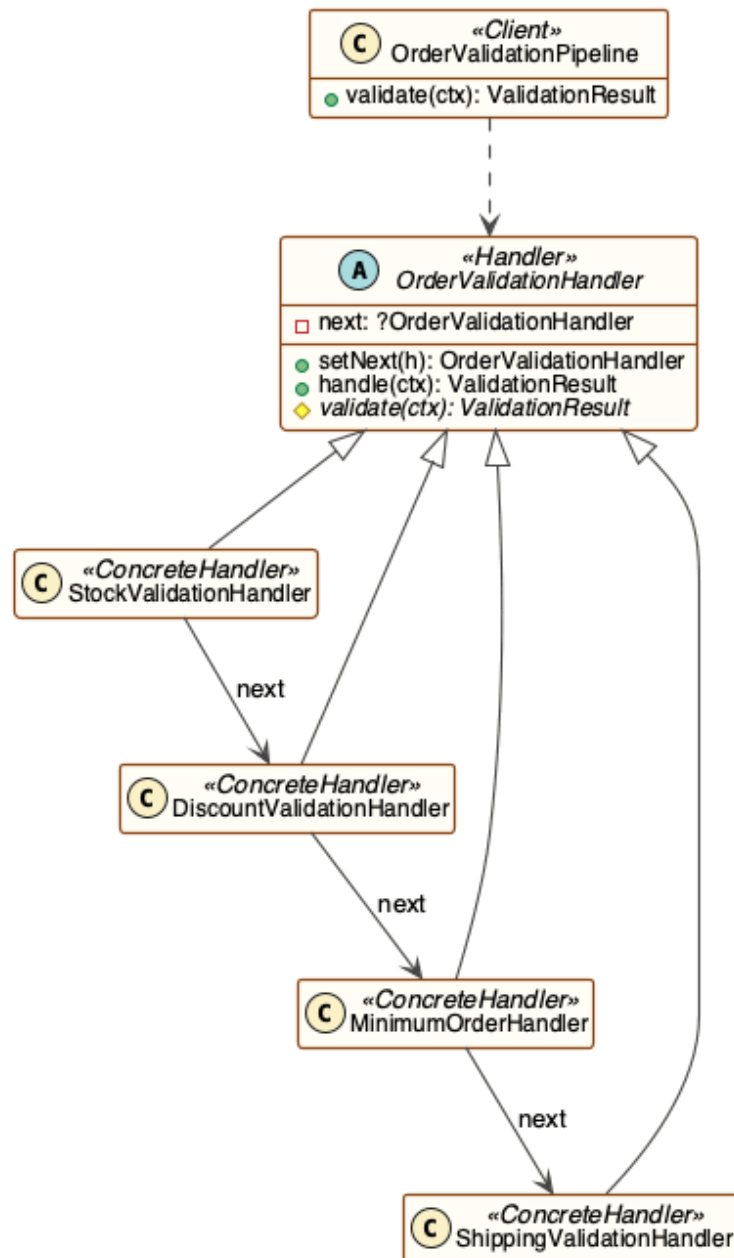


Figura 2.13 – Diagrama UML a pattern-ului Chain of Responsibility

Clasa abstractă `OrderValidationHandler` joacă rolul de `Handler`, cu metoda publică `handle(ctx)` care rulează `validate(ctx)` și, în caz de succes, forwardează apelul la `next`. Concrete Handlers sunt `StockValidationHandler`, `DiscountValidationHandler`, `MinimumOrderHandler` și `ShippingValidationHandler`. Clientul este `OrderValidationPipeline`, care construiește lanțul și îl execută.

`CheckoutFacade` trimite contextul la `OrderValidationPipeline`, care assemblează lanțul în ordinea `Stock → Discount → MinimumOrder → Shipping` și îl pornește. Lanțul se oprește la primul eșec și returnează `ValidationResult` cu mesajul corespunzător. Al patrulea handler, `DiscountValidationHandler`, a fost adăugat ca îmbunătățire recentă: o singură clasă nouă plus o linie în pipeline, fără a atinge validatorii existenți — exact criteriul Open/Closed.

2.4.5 Pattern-ul Template Method

Panoul administrativ Kinder permite exportul listei de comenzi în formate multiple — momentan CSV și JSON, în viitor posibil Excel sau XML. Structura algoritmului de export este identică pentru orice format: se scrie un header, se iterează prin comenzi formatând fiecare rând, se scrie un footer. Ceea ce diferă este doar formatul fiecărei părți: pentru CSV headerul este linia de coloane separată cu virgule, pentru JSON este o paranteză pătrată, iar formatarea unui rând este distinctă.

Pattern-ul Template Method definește scheletul unui algoritm într-o metodă, amânând anumiți pași către subclase. Subclasele pot redefini pașii fără a schimba structura algoritmului. Metoda template este declarată final pentru a interzice override-urile accidentale.

Diagrama UML a componentelor pattern-ului Template Method este prezentată în figura 2.14.

14. Template Method — OrderExportTemplate

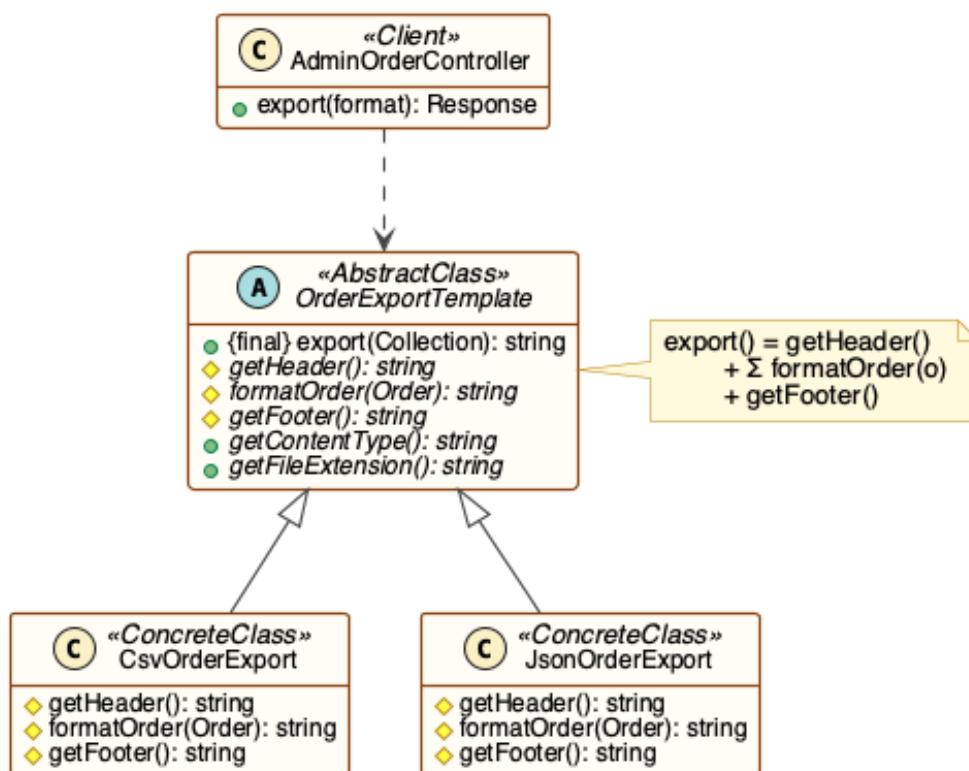


Figura 2.14 – Diagrama UML a pattern-ului Template Method

Clasa abstractă `OrderExportTemplate` joacă rolul de Abstract Class, conținând metoda template final `export(Collection $orders): string` care apelează secvențial `getHeader`, `formatOrder` pentru fiecare comandă și `getFooter`. Concrete Classes sunt `CsvOrderExport` și `JsonOrderExport`, fiecare redefinind cele trei hook-uri plus `getContentType` și `getFileExtension` pentru răspunsul HTTP.

`AdminOrderController::export` primește formatul dorit ca parametru, instanțiază clasa concretă corespunzătoare, apelează `export` pe colecția de comenzi și forțează download-ul cu Content-Type-ul și

extensia potrivite. Adăugarea unui format nou (de exemplu Excel) este o singură clasă nouă — metoda template rămâne neatinsă.

2.4.6 Pattern-ul Observer

Atunci când o comandă este plasată cu succes, trebuie să se întâmple trei acțiuni independente: trimiterea unui email de confirmare către client, actualizarea statisticilor din dashboard-ul administrativ (contor de comenzi, revenue total) și logarea evenimentului pentru audit. Aceste acțiuni sunt decuplate conceptual — dacă logarea eșuează, confirmarea ar trebui să plece oricum. Apelarea lor secvențială din CheckoutFacade ar cupla fațada la toți consumatorii, iar adăugarea unui consumator nou (de exemplu, notificare în Slack pentru echipa de suport) ar necesita modificarea fațadei.

Pattern-ul Observer definește o relație unu-la-mulți între obiecte, astfel încât atunci când un obiect își schimbă starea, toți cei dependenți sunt notificați și actualizați automat.

Diagrama UML a componentelor pattern-ului Observer este prezentată în figura 2.15.

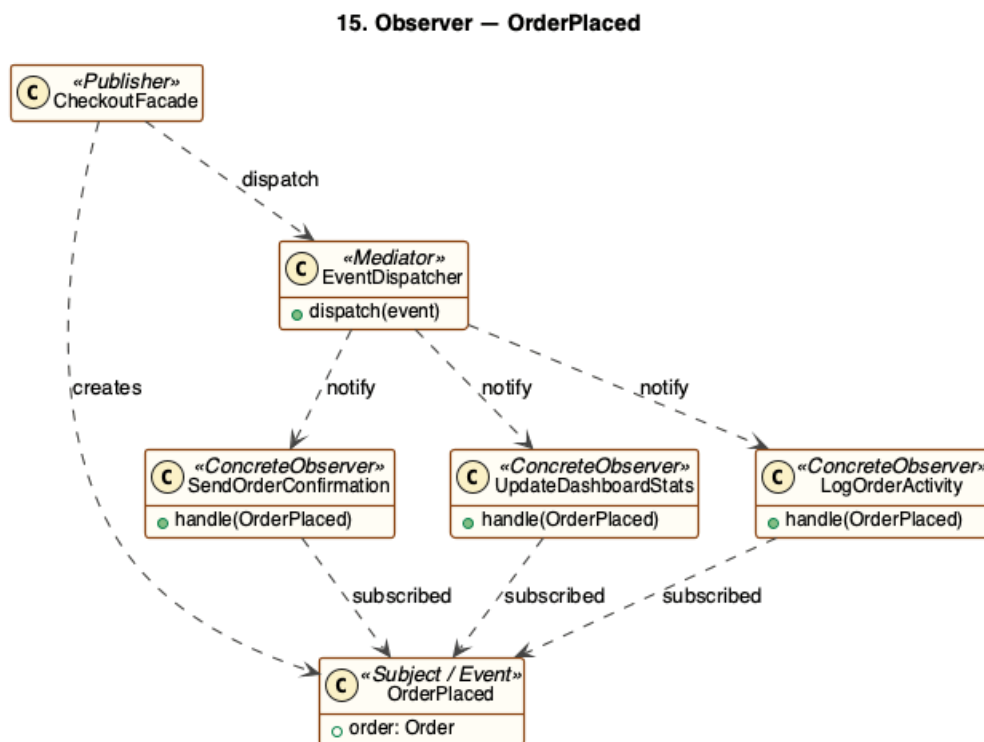


Figura 2.15 – Diagrama UML a pattern-ului Observer

Clasa OrderPlaced joacă rolul de Subject (Event), purtând referința către Order-ul nou creat. Observers (Listeners) sunt SendOrderConfirmation (care folosește intern Abstract Factory-ul de notificări), UpdateDashboardStats și LogOrderActivity. Dispatcher-ul este facade-ul Event al Laravel, care rutează evenimentul către toți ascultătorii înregistrați.

CheckoutFacade emite `event(new OrderPlaced($order))` după persistarea comenzii. `AppServiceProvider::boot` înregistrează cei trei listeners pe evenimentul OrderPlaced. Laravel îi invocă

sincron în ordinea de înregistrare: email de confirmare prin factory-ul injectat, incrementarea statisticilor din cache/DB, log structurat pentru audit. Adăugarea unui observer nou este o singură clasă plus o linie de înregistrare — CheckoutFacade rămâne neschimbată.

3 REALIZAREA APLICAȚIEI

3.1 Stack tehnologic și structura proiectului

Aplicația Kinder a fost realizată folosind stack-ul PHP 8.3 cu framework-ul Laravel 11. Baza de date este SQLite pentru portabilitate — întreaga stare a aplicației se află într-un singur fișier `database.sqlite`, facilitând deployment-ul și testarea. Pentru frontend s-au folosit template-urile Blade ale Laravel împreună cu Tailwind CSS 3 pentru stilizare utilitară și Vite pentru bundling-ul asset-urilor. Managementul dependențelor se face prin Composer (partea PHP) și npm (partea JavaScript).

Structura directoarelor urmează convenția standard Laravel: `app` conține codul aplicației (controllere, modele, servicii, evenimente), `resources/views` conține template-urile Blade, `database/migrations` conține definițiile schemelor iar `public` servește ca document root pentru serverul web. Toate pattern-urile implementate se află în `app/Services` și `app/Notifications`, organizate pe subdomenii. Directorul `docs/uml`, creat special pentru acest raport, conține cele cincisprezece diagrame UML în format PlantUML împreună cu PNG-urile generate.

Aplicația poate fi clonată din repository-ul public și pornită local cu instrucțiunile din README. După rulare, este disponibil un utilizator admin cu parolă implicită care permite testarea fluxurilor administrative. Linkul complet către repository este dat în Anexa A.

3.2 Fluxul de checkout — pattern-urile în acțiune

Fluxul de checkout concentrează cele mai multe pattern-uri lucrând împreună. Utilizatorul începe prin a adăuga produse în coș din paginile catalog sau detaliu produs — fiecare adăugare este un Command (pattern 12) care se stochează în sesiune, permițând undo. Coșul final, ilustrat în figura 3.1, afișează produsele, subtotalul și un buton „Finalizează comanda”.

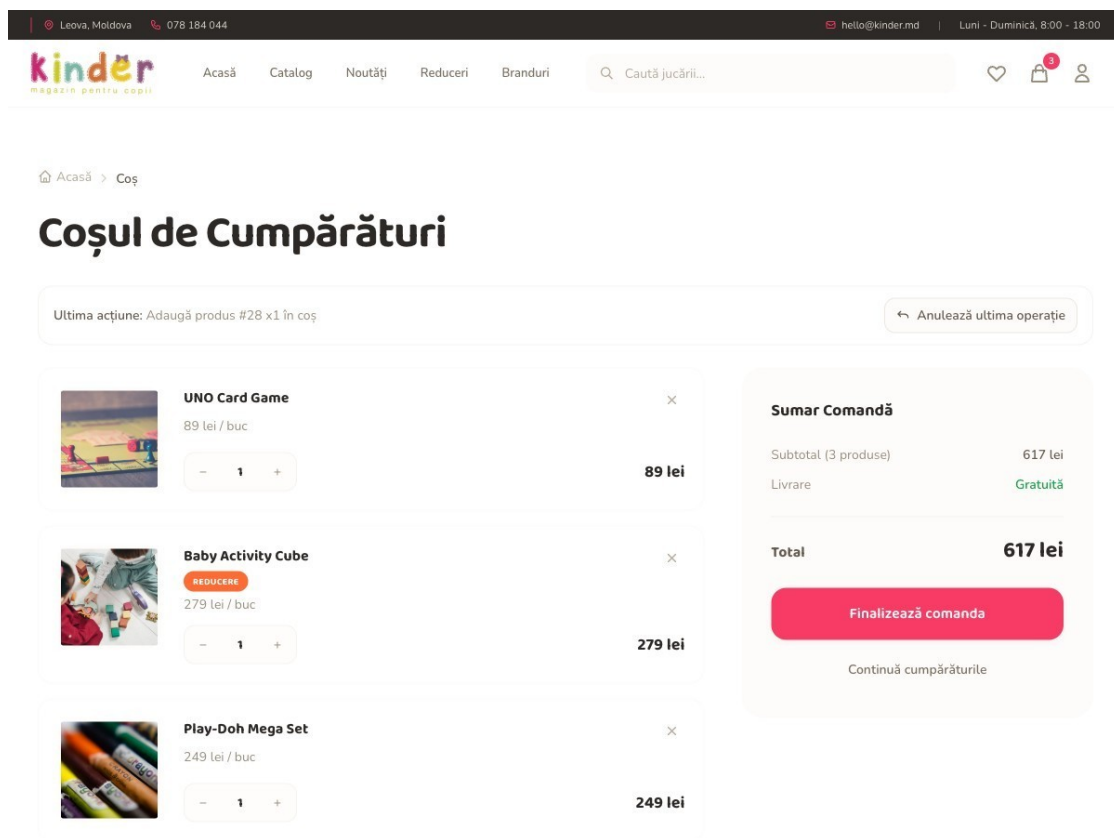


Figura 3.1 – Coșul de cumpărături cu banner de undo activ

Pagina de finalizare comandă, prezentată în figura 3.2, este structurată în cinci secțiuni: informații client, adresa de livrare, metoda de livrare (Standard / Express / Gratuită — Factory Method), metoda de plată (Card / Transfer bancar / Cash on delivery — Adapter) și opțiuni suplimentare (ambalaj cadou — Decorator). La submit, toate datele sunt trimise către CheckoutFacade (Facade), care orchestrează validarea, calculul costurilor, construcția comenzii (Builder) și emiterea evenimentului OrderPlaced (Observer).

Finalizare Comandă

1 Informații Client

NUME COMPLET *
Admin

EMAIL *
admin@kinder.md

TELEFON
+373 XX XXX XXX

2 Adresa de Livrare

ADRESA *
Strada, numărul, apartamentul

ORAȘUL *
Chișinău

COD POȘTAL
ex: MD-2001

3 Metoda de Livrare

☒ **Livrare standard**
2-3 zile lucrătoare 33.00 lei

☐ **Livrare express**
azi sau mâine 89.17 lei

☐ **Livrare gratuită**
5-7 zile lucrătoare **Gratuită**

4 Metoda de Plată

☒ **Plata la livrare**

☐ **Transfer bancar**

☐ **Card bancar**

5 Opțiuni Suplimentare

☐ **Ambalare cadou**
Ambalaj festiv cu fundiță +25 lei

NOTE COMANDĂ (OPTIONAL)
Instrucțiuni speciale pentru livrare...

Sumar Comandă

UNO Card Game Cantitate: 1	89 lei
Baby Activity Cube Cantitate: 1	279 lei
Play-Doh Mega Set Cantitate: 1	249 lei
Subtotal	617 lei
Livrare	Gratuită
Total	617 lei

Plasează comanda

Plată securizată Date originale

Figura 3.2 – Pagina de finalizare comandă — orchestrată integral de CheckoutFacade

Validarea înainte de plasare trece comanda prin lanțul Stock → Discount → MinimumOrder → Shipping (Chain of Responsibility). Dacă toate validările trec, OrderBuilder construiește obiectul Order într-o singură tranzacție DB, aplicând pe parcurs decoratorii de preț și decrementând stocurile. Rezultatul, ilustrat în figura 3.3, este o pagină de confirmare cu numărul unic al comenzii, lista produselor și totalul.

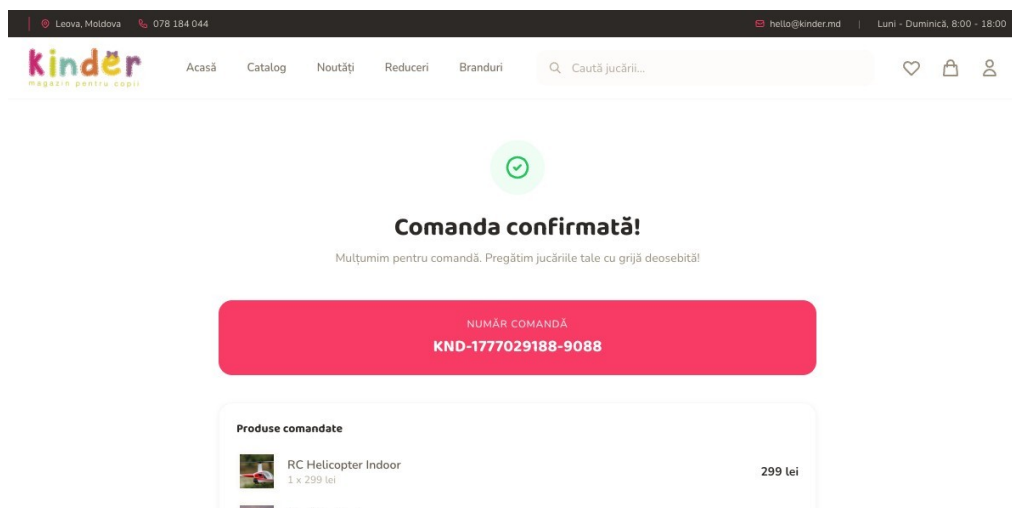


Figura 3.3 – Pagina de confirmare — rezultatul apelului Builder

3.3 Panoul administrativ

Panoul administrativ oferă acces la managementul întregii aplicații. Dashboard-ul, ilustrat în figura 3.4, afișează statistici sumare — numărul total de comenzi, revenue-ul cumulat, numărul produselor și utilizatorilor, plus lista comenzilor recente și un alert pentru stoc scăzut. Actualizările dashboard-ului sunt efectuate de Listener-ul UpdateDashboardStats (Observer, pattern 15), care reacționează imediat la evenimentul OrderPlaced.

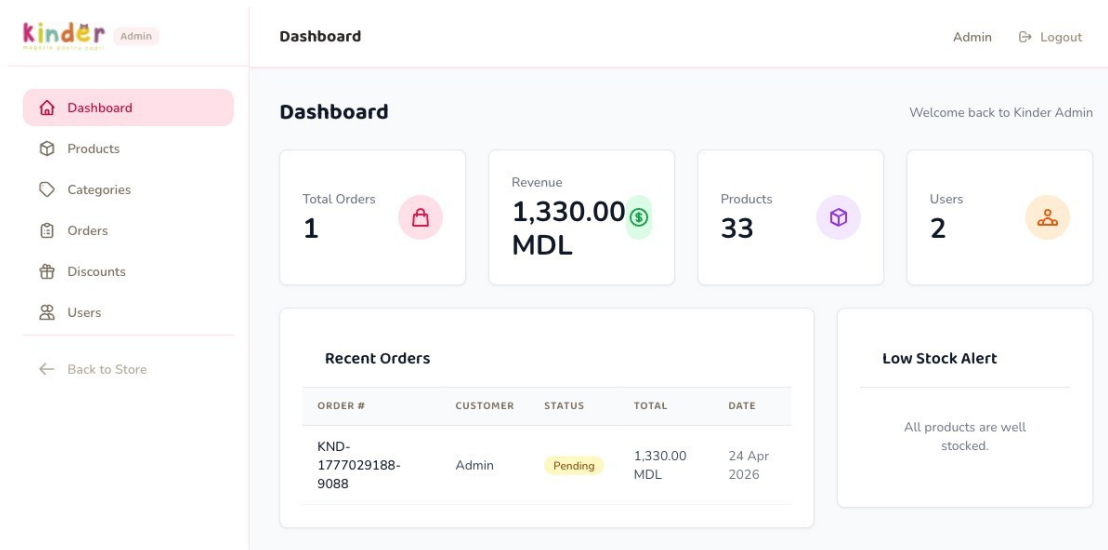


Figura 3.4 – Dashboard administrativ — actualizat prin Observer

Pagina de produse, prezentată în figura 3.5, oferă căutare, paginare și pentru fiecare produs trei acțiuni: editare, duplicare (Prototype, pattern 2) și ștergere. Butonul de duplicare declanșează `Product::duplicate`, care creează o copie inactivă cu SKU nou și imagini recopiate — administratorul poate astfel genera variante de produs fără a re-introduce cele douăzeci de câmpuri.


Admin

Dashboard
Products
Categories
Orders
Discounts
Users
Back to Store

Dashboard

Products

Categories

Orders

Discounts

Users

Back to Store

Admin

Logout

Products

+

Add Product

Search products by name, SKU, or brand...

Search

IMAGE	NAME	CATEGORY	PRICE	STOCK	STATUS	ACTIONS
	Soft Rattle Set 4pc SKU: KND-3RDBAK6I NEW	Rattles	129.00 MDL	30	Active	Edit Delete
	Musical Baby Walker SKU: KND-KOXMCDY	Baby Toys	549.00 MDL	10	Active	Edit Delete
	Stacking Rings Classic SKU: KND-CP7B40DY	Baby Toys	79.00 MDL	40	Active	Edit Delete
	Baby Activity Cube SKU: KND-QHZA2CGM HOT	Baby Toys	399.00 MDL 299.00 MDL	15	Active	Edit Delete
	Jewelry Making Kit SKU: KND-LJLUGU4 NEW	Arts & Crafts	199.00 MDL	20	Active	Edit Delete
	Play-Doh Mega Set SKU: KND-PR8R2VVG	Arts & Crafts	249.00 MDL	30	Active	Edit Delete
	Kids Art Set 150 Pieces SKU: KND-RP99CUD5	Arts & Crafts	349.00 MDL 249.00 MDL	28	Active	Edit Delete
	RC Helicopter Indoor SKU: KND-QYFGS2RN	Remote Control	299.00 MDL	15	Active	Edit Delete
	RC Drone with Camera SKU: KND-6LCOWBOK NEW	Remote Control	799.00 MDL	7	Active	Edit Delete
	RC Racing Car 1:16 SKU: KND-HP1CLLC HOT	Remote Control	549.00 MDL 449.00 MDL	12	Active	Edit Delete
	Disney Princess Puzzle 100pc SKU: KND-KBDGKWH3 NEW	Puzzles	99.00 MDL	25	Active	Edit Delete
	World Map Puzzle 500pc SKU: KND-BNGI5DWM	Puzzles	179.00 MDL	18	Active	Edit Delete
	Water Blaster Super Soaker SKU: KND-KJ7FKTOU	Outdoor Toys	249.00 MDL	20	Active	Edit Delete
	Bubble Machine Deluxe SKU: KND-57HYCW4U HOT	Outdoor Toys	199.00 MDL 149.00 MDL	25	Active	Edit Delete
	Kids Bicycle 16 inch SKU: KND-NNPLO6BN	Outdoor Toys	1,499.00 MDL	8	Active	Edit Delete

Showing 1 to 15 of 33 results

<

1

2

3

>

Figura 3.5 – Lista de produse din panoul admin

Pagina de detaliu a unei comenzi, prezentată în figura 3.6, folosește pattern-ul State (pattern 11) pentru a afișa în dropdown-ul „Update Status” doar tranzițiile legale din starea curentă — de exemplu, dintr-o comandă „pending” se poate merge în „processing” sau „cancelled”, dar nu direct în „delivered”. La schimbarea statusului, acțiunile specifice (trimiterea notificării, setarea tracking-ului) sunt executate de starea concretă.

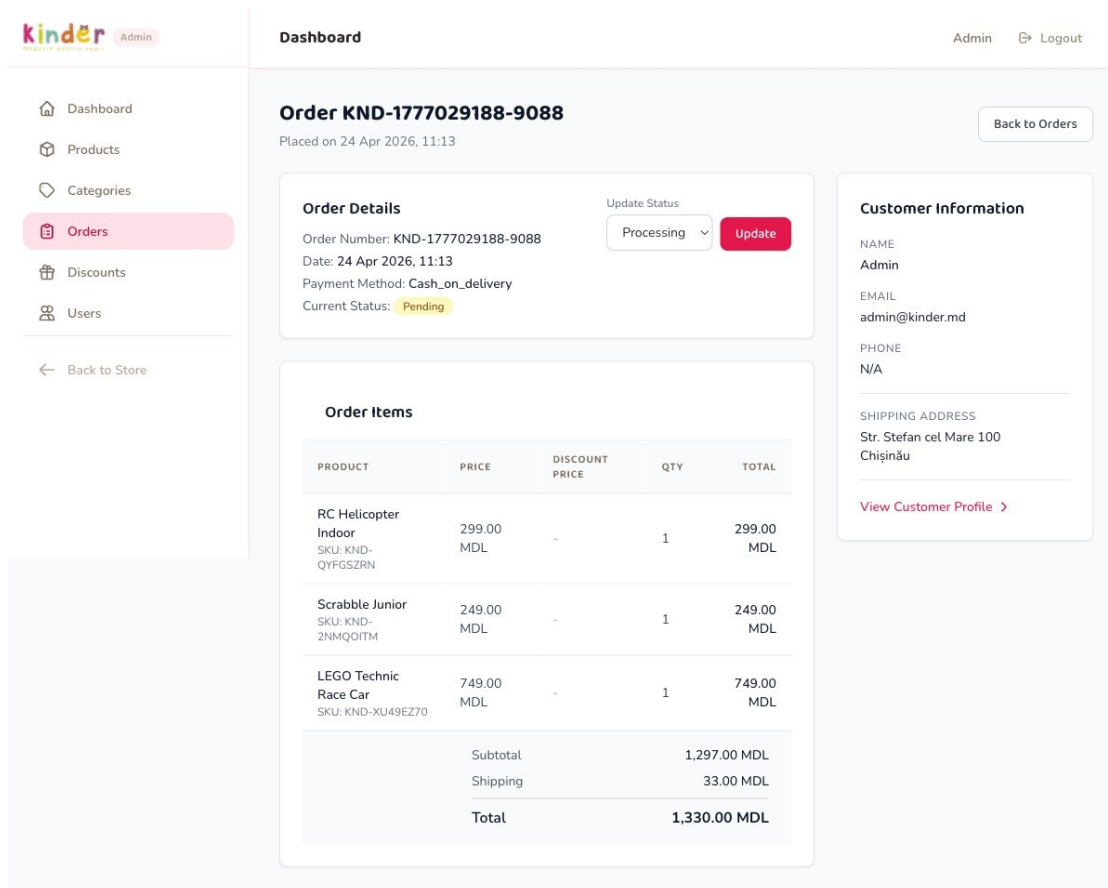


Figura 3.6 – Pagina de detaliu comandă cu State Pattern

3.4 Extensibilitatea demonstrată

Dincolo de funcționalitatea inițială, proiectul a primit pe parcurs două îmbunătățiri care validează în practică Principiul Open/Closed. Prima adaugă un al treilea canal de notificare — notificări Push, potrivite pentru o aplicație mobilă — sub forma clasei `PushNotificationFactory`. Clasa implementează interfața `OrderNotificationFactory` existentă, producând variantele short-form ale celor trei notificări (confirmare, status, expediere), cu text concis adaptat conștrângerilor lockscreen-ului. Nicio altă parte a codului nu a fost modificată; factory-ul `Push` este complet interschimbabil cu Email-ul și SMS-ul existente.

A doua îmbunătățire adaugă un al patrulea handler în lanțul de validare — `DiscountValidationHandler`. Acest validator compară prețul stocat în coșul utilizatorului cu prețul real curent al produsului (calculat ținând cont de discount-urile active), respingând comanda dacă discount-ul a expirat între momentul adăugării în coș și apăsarea butonului de checkout. Integrarea a fost o singură clasă nouă plus o linie suplimentară în `OrderValidationPipeline`, fără a modifica validatorii `Stock`, `MinimumOrder` sau `Shipping`.

Ambele modificări demonstrează concret ceea ce teoria promite: un sistem bine structurat pe pattern-uri GoF acceptă extinderea prin adăugare, nu prin modificare. Costul marginal al adăugării unei funcționalități noi rămâne mic chiar și pe măsură ce aplicația crește, iar riscul de regresie se reduce considerabil pentru că codul existent nu este atins.

CONCLUZII

Proiectul de an realizat a atins integral obiectivele propuse. Platforma de comerț electronic Kinder a fost construită funcțional, cu toate fluxurile esențiale ale unui magazin online: catalog organizat ierarhic pe categorii, coș cu istoric și undo, checkout orchestrat cu validare, calcul de shipping, procesare plată și notificări, panou administrativ pentru gestiunea completă a produselor, categoriilor, comenzilor, reducerilor și utilizatorilor. Interacțiunea vizuală este polișă prin Tailwind CSS, iar navigarea este intuitivă atât pentru utilizatorul final cât și pentru administrator.

În ceea ce privește obiectivul didactic central — aplicarea pattern-urilor GoF — minimul cerut de disciplină (douăsprezece pattern-uri, câte patru din fiecare categorie) a fost nu doar atins, ci depășit: aplicația implementează cincisprezece pattern-uri, patru creaționale, cinci structurale și șase comportamentale. Fiecare pattern a fost selectat pentru că rezolvă o problemă reală a domeniului, nu pentru a bifa un item dintr-o listă. Builder-ul gestionează complexitatea reală a construcției unei comenzi. Abstract Factory-ul produce familii de notificări per canal cu structuri distincte. Composite-ul modelează un arbore de categorii cu adâncime variabilă. Chain of Responsibility-ul lanțuie validările într-o ordine corectă.

Două îmbunătățiri ulterioare — adăugarea canalului Push și a validatorului de discount-uri — au arătat în practică valoarea arhitecturii pe pattern-uri. Ambele au fost adaus fără modificarea codului existent, respectând exact Principiul Open/Closed. Acest experiment mic dar concret demonstrează că alegerile de design făcute inițial plătesc dividende pe termen lung.

Pe plan tehnologic, lucrul cu Laravel 11 a confirmat sinergia dintre pattern-urile aplicate la nivelul aplicației și cele existente în framework. Facade-urile, containerul de dependențe, sistemul de evenimente și middleware-ul au oferit o infrastructură consistentă pe care pattern-urile proprii s-au așezat natural. Această observație validează o bună practică frecvent invocată în comunitatea profesională: un framework bine conceput economisește dezvoltatorului reinventarea roții.

Limitările recunoscute ale proiectului țin de amploare: nu au fost implementate toate cele douăzeci și trei de pattern-uri GoF (Singleton, Mediator, Flyweight, Memento, Visitor, Interpreter, Bridge, Iterator nu sunt prezente pentru că domeniul ales nu generează natural nevoia lor). Integrarea cu gateway-uri reale de plată este simulată prin mock-uri (StripeGatewayClient, LocalBankApiClient) — validarea Adapter-ului este structurală, nu end-to-end. Testele automate acoperă fluxurile principale dar nu toate ramurile alternative.

Dirjecțiile viitoare de dezvoltare includ integrarea reală a unui gateway de plată, adăugarea unui sistem de recomandări bazat pe istoricul utilizatorului (care ar putea ilustra pattern-ul Iterator sau Visitor peste catalogul de produse), portarea notificărilor push către un serviciu real (Firebase Cloud Messaging), precum și extinderea suitei de teste automate cu cazuri de edge pentru fiecare pattern.

În concluzie, acest proiect a fost o experiență formatoare de sinteză: a pornit de la un caz concret de domeniu, a aplicat un set extins de pattern-uri clasice și a arătat că acestea își păstrează validitatea tehnică și valoarea didactică într-un context real de dezvoltare software modernă.

BIBLIOGRAFIE

- [1] E. GAMMA, R. HELM, R. JOHNSON, J. VLISSIDES. Design Patterns: Elements of Reusable Object-Oriented Software. Boston: Addison-Wesley, 1994. 395 p. ISBN 0-201-63361-2.
- [2] Refactoring.Guru. Design Patterns [online]. [citat 20.04.2026]. Disponibil: <https://refactoring.guru/design-patterns/catalog>
- [3] Laravel Documentation [online]. © Laravel LLC, 2026 [citat 22.04.2026]. Disponibil: <https://laravel.com/docs/11.x>
- [4] R. MARTIN. Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River: Prentice Hall, 2008. 464 p. ISBN 978-0-13-235088-4.
- [5] R. MARTIN. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston: Prentice Hall, 2017. 432 p. ISBN 978-0-13-449416-6.
- [6] PHP: Hypertext Preprocessor [online]. © The PHP Group, 2026 [citat 20.04.2026]. Disponibil: <https://www.php.net/manual/en/>
- [7] Eloquent: Getting Started — Laravel Documentation [online]. [citat 22.04.2026]. Disponibil: <https://laravel.com/docs/11.x/eloquent>
- [8] Tailwind CSS Documentation [online]. © Tailwind Labs, 2026 [citat 20.04.2026]. Disponibil: <https://tailwindcss.com/docs>
- [9] OMG Unified Modeling Language (OMG UML). Version 2.5.1 [online]. Object Management Group, 2017 [citat 20.04.2026]. Disponibil: <https://www.omg.org/spec/UML/2.5.1>
- [10] M. FOWLER. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002. 533 p. ISBN 0-321-12742-0.
- [11] PlantUML Language Reference Guide [online]. [citat 23.04.2026]. Disponibil: <https://plantuml.com/guide>
- [12] SQLite Documentation [online]. © SQLite Developers, 2026 [citat 20.04.2026]. Disponibil: <https://www.sqlite.org/docs.html>
- [13] CHIRCU, Mihail. Kinder — magazin online pentru copii. Cod sursă [online]. 2026 [citat 24.04.2026]. Disponibil: <https://github.com/mihail-chircu/TMPPP>

ANEXA A

Repository GitHub și structura proiectului

Codul sursă complet al aplicației Kinder este disponibil public pe GitHub, sub directorul „proiect final” al repository-ului TMPPP, împreună cu toate lucrările de laborator din cadrul disciplinei Tehnici Moderne de Programare pentru Platforme. Repository-ul conține întreaga bază de cod, migrările bazei de date, seed-urile pentru populare cu date de test, cele cincisprezece diagrame UML în format PlantUML și PDF-ul ilustrativ al pattern-urilor.

Link repository: <https://github.com/mihail-chircu/TMPPP>

Proiectul este organizat după convenția standard Laravel. Directoarele principale de interes pentru pattern-urile studiate sunt:

- app/Services — layer-ul de business cu subdomeniile Cart, Catalog, Checkout, Export, Order, OrderState, Payment, Pricing, Shipping, Validation;
- app/Notifications — arhitectura Abstract Factory pentru notificări multi-canal (Email, SMS, Push);
- app/Repositories — ProductRepository și CachingProductProxy (Proxy Pattern);
- app/Events și app/Listeners — evenimentul OrderPlaced și cei trei ascultători (Observer);
- app/Contracts — interfețele partajate (Prototype, CategoryComponent, ProductRepositoryInterface);
- docs/uml — sursele PlantUML și PNG-urile celor cincisprezece diagrame;
- docs/screenshots — capturile de ecran utilizate în prezentul raport și în PDF-ul ilustrativ.

Pentru rulare locală, este suficientă clonarea repository-ului, instalarea dependențelor prin composer install și npm install, configurarea fișierului .env și rularea migrărilor prin php artisan migrate --seed. Detalii suplimentare se găsesc în README.md.

ANEXA B

Fragmente de cod reprezentativ

Anexa prezintă fragmente de cod selectate din aplicație, ilustrând concret implementarea pattern-urilor descrise în Capitolul 2. Pentru completitudine, sursele integrale se găsesc în repository-ul GitHub indicat în Anexa A.

B.1 OrderBuilder — pattern-ul Builder

Mai jos este reprodus un fragment din clasa OrderBuilder, arătând metoda build() care finalizează construcția comenzii într-o tranzacție atomică.

```
public function build(): Order
{
    return DB::transaction(function () {
        $priceCalc = $this->buildPrice($this->data['subtotal']);
        $breakdown = $priceCalc->getBreakdown();
        $total = $priceCalc->calculate() + $this->data['shipping_cost'];

        $order = Order::create([
            'order_number' => $this->generateOrderNumber(),
            'user_id' => $this->data['user_id'],
            'customer_name' => $this->data['customer_name'],
            'customer_email' => $this->data['customer_email'],
            'shipping_address' => $this->data['shipping_address'],
            'shipping_city' => $this->data['shipping_city'],
            'payment_method' => $this->data['payment_method'],
            'shipping_method' => $this->data['shipping_method'],
            'shipping_cost' => $this->data['shipping_cost'],
            'subtotal' => $this->data['subtotal'],
            'total' => $total,
            'status' => 'pending',
            'notes' => $this->data['notes'] ?? null,
        ]);

        foreach ($this->data['cart_items'] as $item) {
            OrderItem::create([
                'order_id' => $order->id,
                'product_id' => $item->product_id,
                'product_name' => $item->product->name,
                'quantity' => $item->quantity,
                'price' => $item->price,
                'total' => $item->price * $item->quantity,
            ]);
            Product::where('id', $item->product_id)
                ->decrement('stock', $item->quantity);
            Product::where('id', $item->product_id)
                ->increment('sales_count', $item->quantity);
        }

        return $order;
    });
}
```

Logica colaterală (tranzacția, decrementarea stocurilor, incrementarea contoarelor de vânzări) este încapsulată în metoda finală `build`, iar asamblarea propriu-zisă a datelor se face prin apelurile fluent ale metodelor `with*`.

B.2 CachingProductProxy — pattern-ul Proxy

Fragmentul arată modul în care proxy-ul împachetează fiecare metodă a repository-ului real într-un apel `Cache::remember` cu TTL de 30 de minute.

```
class CachingProductProxy implements ProductRepositoryInterface
{
    private const TTL = 60 * 30; // 30 minutes

    public function __construct(
        private ProductRepository $repo,
    ) {}

    public function getFeatured(): Collection
    {
        return Cache::remember(
            'products:featured',
            self::TTL,
            fn () => $this->repo->getFeatured(),
        );
    }

    public function getNew(): Collection
    {
        return Cache::remember(
            'products:new',
            self::TTL,
            fn () => $this->repo->getNew(),
        );
    }
    // ... getSale și getPopular similar
}
```

Binding-ul se face în `AppServiceProvider` prin linia `$this->app->bind(ProductRepositoryInterface::class, CachingProductProxy::class)`, iar `HomeController` primește interfața prin injecție de dependență, fără să știe că rezolvarea concretă este un proxy.